

UNIVERSITAT POLITÈCNICA DE VALÈNCIA

etsinf

Escola Tècnica Superior d'Enginyeria Informàtica

Escuela Técnica Superior de Ingeniería Informática
Universidad Politécnica de Valencia

De señal acústica a partitura: transcripción automática de piano mediante aprendizaje profundo

TRABAJO FIN DE GRADO

Grado en Ingeniería Informática

Autor: Jorge Rodríguez González

Tutor: Alfons Juan Císcar

Curso 2025-2026

Resum

La transcripció automàtica de música (AMT) constitueix un dels problemes centrals de la recuperació d'informació musical (MIR), l'objectiu de la qual és convertir un senyal d'àudio en una representació simbòlica de la música interpretada. En el cas del piano, l'objectiu és obtindre a partir de l'àudio una representació MIDI que capture amb precisió els esdeveniments de nota –to, inici, final i dinàmica– presents en la interpretació. Aquest treball aborda la replicació i l'avaluació de dos sistemes de l'estat de l'art per a la transcripció automàtica. En primer lloc, s'entrena un model MFT-FiLM (*Multitask Frequency-Time Feature-wise Linear Modulation*) que combina convolucions espectrals i xarxes recurrents bidireccionals per a l'estimació conjunta d'*onsets*, *offsets*, *frames* i velocitats. A continuació, s'entrena un model Transkun, un sistema basat en *transformers* que aborda la transcripció com un problema de traducció de seqüències. Tots dos models s'avaluen de forma quantitativa sobre el conjunt de test de MAESTRO seguint el protocol estàndard (`mir_eval`). Addicionalment, s'avalua la capacitat de generalització de tots dos models sobre el *dataset* PiJAMA, la distribució de repertori i àudio del qual difereix de MAESTRO, i s'exploren modificacions experimentals orientades a la millora del rendiment dels models.

Paraules clau: Transcripció automàtica, xarxes convolucionals, *transformers*, *onset*, *offset*, *frame*, modulació lineal, MIDI, espectrograma, MAESTRO, PiJAMA, piano

Resumen

La transcripción automática de música (AMT) constituye uno de los problemas centrales de la recuperación de información musical (MIR), cuyo objetivo es convertir una señal de audio en una representación simbólica de la música interpretada. En el caso del piano, el objetivo es obtener a partir del audio una representación MIDI que capture con precisión los eventos de nota –tono, inicio, final y dinámica– presentes en la interpretación. Este trabajo aborda la replicación y evaluación de dos sistemas del estado del arte para la transcripción automática. En primer lugar, se entrena un modelo MFT-FiLM (*Multitask Frequency-Time Feature-wise Linear Modulation*) que combina convoluciones espectrales y redes recurrentes bidireccionales para la estimación conjunta de *onsets*, *offsets*, *frames* y velocidades. A continuación, se entrena un modelo Transkun, un sistema basado en transformers que aborda la transcripción como un problema de traducción de secuencias. Ambos modelos se evalúan de forma cuantitativa sobre el conjunto de test de MAESTRO siguiendo el protocolo estándar (`mir_eval`). Adicionalmente se evalúa la capacidad de generalización de ambos modelos sobre el *dataset* PiJAMA, cuya distribución de repertorio y audio difiere de MAESTRO, y se exploran modificaciones experimentales orientadas a la mejora del rendimiento de los modelos.

Palabras clave: Transcripción automática, redes convolucionales, *transformers*, *onset*, *offset*, *frame*, modulación lineal, MIDI, espectrograma, MAESTRO, PiJAMA, piano

Abstract

Automatic Music Transcription (AMT) constitutes one of the core problems in Music Information Retrieval (MIR), aiming to convert an audio signal into a symbolic representation of the performed music. In the case of the piano, the goal is to obtain a MIDI representation from the audio that accurately captures the note events –pitch, onset, offset, and dynamics– present in the performance. This work addresses the replication and evaluation of two state-of-the-art systems for automatic transcription. First, an MFT-FiLM

(Multitask Frequency-Time Feature-wise Linear Modulation) model is trained, which combines spectral convolutions and bidirectional recurrent networks for the joint estimation of onsets, offsets, frames, and velocities. Next, a Transkun model is trained, a *transformer*-based system that approaches transcription as a sequence translation problem. Both models are quantitatively evaluated on the MAESTRO test set following the standard protocol (`mir_eval`). Additionally, the generalization capability of both models is evaluated on the PiJAMA *dataset*, whose repertoire and audio distribution differs from MAESTRO, and experimental modifications aimed at improving the models' performance are explored.

Key words: Automatic transcription, convolutional networks, *transformers*, *onset*, *offset*, *frame*, linear modulation, MIDI, spectrogram, MAESTRO, PiJAMA, piano

Índice general

Índice general v
Índice de figuras vii
Índice de tablas vii

Lista de Acrónimos ix
1 Introducción 1
1.1 Motivación . . . 1
1.2 Objetivos . . . 1
1.3 Estructura de la memoria . . . 1
2 Preliminares 3
2.1 ?? ???? ???? ? ?? ?? . . . 3
3 MAESTRO 5
3.1 Introducción . . . 5
3.2 Origen de los datos . . . 5
3.3 Adquisición y anotación . . . 6
3.4 Características del *dataset* . . . 7
3.5 Evolución del *dataset* . . . 9
3.6 Distribución y licencia . . . 9
3.7 Limitaciones . . . 9
3.8 Relevancia para este trabajo . . . 10
3.9 Evaluación estándar . . . 10
4 MFT-FiLM 13
4.1 Introducción . . . 13
4.2 Arquitectura del modelo . . . 14
4.3 Función de pérdida y eficiencia . . . 17
4.4 Configuración del experimento . . . 18
4.5 Preproceso y entrenamiento . . . 22
4.6 Resultados . . . 23
5 Transkun 29
5.1 Introducción . . . 29
5.2 Arquitectura del modelo . . . 30
5.3 *Scoring* por producto interior y semi-CRF . . . 34
5.4 Predicción de atributos y procesado por segmentos . . . 36
5.5 Configuración del experimento . . . 38
5.6 Preproceso y entrenamiento . . . 38
5.7 Resultados . . . 38
6 Conclusiones 39
Bibliografía 41

Apéndices
A Configuració del sistema 43

A.1 Fase d'inicialització 43
A.2 Identificació de dispositius 43
B ??? ???????????? ???? 45

Índice de figuras

3.1 Sistema de sensores de un Yamaha Disklavier 6

4.1 Comparación de representaciones tiempo–frecuencia 14
4.2 Esquema general del modelo MFT-FiLM 15
4.3 Evolución de la pérdida (*loss*) del modelo MFT-FiLM 25
4.4 Evolución de la métrica F_1 en el conjunto de validación 26
4.5 Comparativa entre los niveles de precisión y *recall* en el conjunto de validación 26

5.1 Esquema general del modelo Transkun 31
5.2 Bloques del transformer encoder de TranskUN 33

Índice de tablas

3.1 Top 20 compositores en MAESTRO V3.0.0 8
3.2 Estadísticas de MAESTRO V3.0.0 por partición 8
3.3 Evolución de MAESTRO entre las versiones V1, V2 y V3 9
3.4 Condiciones de aceptación de un evento predicho como verdadero positivo según la variante de métrica de nota empleada. 12

4.1 Configuración de las capas de salida de cada rama del sistema MFT-FiLM. 17
4.2 Parámetros de configuración establecidos en `constant_macro.py` para el entrenamiento de MFT-FILM. 21
4.3 Resultados del F_1 de MFT-FiLM sobre MAESTRO v3.0.0 24
4.4 Resultados de la precisión de MFT-FiLM sobre MAESTRO v3.0.0 24
4.5 Resultados del *recall* de MFT-FiLM sobre MAESTRO v3.0.0 24

Lista de Acrónimos

AMT *Automatic Music Transcription* (Transcripción Automática de Música)

ASAP *Aligned Scores and Performances*

CC *Control Change* (mensaje de control MIDI)

CQT *Constant-Q Transform* (Transformada Q Constante)

CSV *Comma-Separated Values*

MEI *Music Encoding Initiative*

MusicXML *Music Extensible Markup Language*

FiLM *Feature-wise Linear Modulation* (Modulación Lineal por Características)

FG-LSTM *Frequency-Grouped Long Short-Term Memory*

FT-FiLM *Frequency-Time Feature-wise Linear Modulation*

IFT-FiLM *Inter-Task Frequency-Time Feature-wise Linear Modulation*

JSON *JavaScript Object Notation*

LSTM *Long Short-Term Memory*

MAESTRO *MIDI and Audio Edited for Synchronous Tracks and Organization*

MAPS *MIDI Aligned Piano Sounds*

MEI *Music Encoding Initiative*

MFT-FiLM *Multitask Frequency-Time Feature-wise Linear Modulation*

MIDI *Musical Instrument Digital Interface*

MIR *Music Information Retrieval* (Recuperación de Información Musical)

MTL *Multitask Learning* (Aprendizaje Multitarea)

PCM *Pulse Code Modulation* (Modulación por Impulsos Codificados)

PiJAMA *Piano Jazz and More Audio*

SLURM *Simple Linux Utility for Resource Management*

STFT *Short-Time Fourier Transform* (Transformada de Fourier de Tiempo Corto)

VRAM *Video Random Access Memory*

WAV *Waveform Audio File Format*

BCE *Binary Cross-Entropy* (Entropía Cruzada Binaria)

CCE *Categorical Cross-Entropy* (Entropía Cruzada Categórica)

MAC *Multiply–Accumulate* (Operación de Multiplicación-Accumulación)

ReLU *Rectified Linear Unit*

GELU *Gaussian Error Linear Unit*

SGD *Stochastic Gradient Descent* (Descenso de Gradiente Estocástico)

ECC *Error-Correcting Code*

HPC *High-Performance Computing* (Cómputo de Altas Prestaciones)

CAPÍTULO 1
Introducción

????? ????????????? ????????????? ????????????? ????????????? ?????????????

1.1 Motivación

????? ????????????? ????????????? ????????????? ????????????? ?????????????

1.2 Objetivos

????? ????????????? ????????????? ????????????? ????????????? ?????????????

1.3 Estructura de la memoria

????? ????????????? ????????????? ????????????? ????????????? ?????????????

CAPÍTULO 2
Preliminares

????? ????????????? ????????????? ????????????? ????????????? ?????????????

2.1 ?? ???? ???? ? ?? ??

????? ????????????? ????????????? ????????????? ????????????? ?????????????

CAPÍTULO 3
MAESTRO

3.1 Introducción

Uno de los principales factores que limitan el rendimiento de los sistemas de AMT basados en el aprendizaje profundo es la disponibilidad de datos de entrenamiento etiquetados con alta precisión y en cantidad suficiente para que los modelos sean capaces de generalizar. Hasta 2018, los *datasets* disponibles para esta tarea, como **MAPS** (*MIDI Aligned Piano Sounds*), no superaban las 70 horas de audio, lo que suponía una barrera significativa para el entrenamiento de nuevos modelos [1].

En el ámbito de este trabajo, **Google Magenta** publicó el *dataset* **MAESTRO** [6] (*Midi and Audio Edited for Synchronous Tracks and Organization*), que supuso un cambio de paradigma al ofrecer casi 200 horas de grabaciones de piano de alta calidad con alineación precisa entre audio y etiquetas MIDI. La publicación de MAESTRO demostró inmediatamente su impacto: el modelo **Onsets and Frames**, entrenado sobre MAESTRO alcanzó un **F1 de 67,43 %** en la evaluación sobre grabaciones Disklavier del conjunto MAPS, superando notablemente el **50,22 %** obtenido por el mismo modelo entrenado sobre MAPS únicamente [5, 6]. Desde entonces, MAESTRO se ha consolidado como el estándar de facto para la evaluación y comparación de sistemas de transcripción automática de piano.

En este trabajo se utiliza la versión **V3.0.0** [3] del *dataset*, publicada en 2020, que constituye una versión más depurada y es actualmente la referencia en la comunidad investigadora.

Este capítulo analiza el *dataset* MAESTRO. La Sección 3.2 describe su origen en la *International Piano-e-Competition* y el uso de pianos Yamaha Disklavier. El proceso de adquisición y anotación se detalla en la Sección 3.3, mientras que sus características técnicas, evolución y licenciamiento se exponen en las Secciones 3.4, 3.5 y 3.6, respectivamente. Posteriormente, se discuten sus limitaciones (Sección 3.7) y su relevancia para este trabajo (Sección 3.8). Finalmente, la Sección 3.9 presenta el protocolo de evaluación estándar para la transcripción automática de piano.

3.2 Origen de los datos

Los datos que constituyen MAESTRO provienen de grabaciones recogidas durante diez ediciones de la *International Piano-e-Competition*, una competición de piano de nivel internacional celebrada anualmente en la que pianistas virtuosos interpretan sobre pianos de cola **Yamaha Disklavier** equipados con un sistema integrado de captura MIDI de alta precisión [3]. Esta competición es la fuente exclusiva de todas las grabaciones del

dataset, lo que dota al conjunto de datos de una coherencia excepcional en términos de calidad de la interpretación, condiciones acústicas y equipamiento.

En cuanto al instrumento empleado para la captación, el elemento técnico que garantiza la calidad de MAESTRO es el piano Yamaha Disklavier, un piano de cola acústico convencional al que se le ha añadido un sistema electrónico de captura que registra, con alta precisión temporal, el movimiento de cada tecla y pedal durante la interpretación [6]. Este sistema está formado por un conjunto de sensores situados bajo las teclas y pedales, que detectan la posición y velocidad de cada elemento, así como actuadores que permiten reproducir automáticamente la interpretación a partir de los datos MIDI [4]. A diferencia de los sintetizadores o controladores MIDI, el Disklavier produce sonido acústico real al mismo tiempo que genera los datos MIDI, lo que garantiza que el audio captado por los micrófonos y el MIDI registrado sean inherentemente sincrónicos [6].

La Figura 3.1 ilustra la disposición de estos componentes bajo el teclado. La clave tecnológica que aporta valor a este *dataset* reside en el uso de **sensores ópticos sin contacto**. Al evitar cualquier fricción o resistencia mecánica adicional, el sistema logra escanear continuamente el movimiento del martillo y la tecla para medir la dinámica (velocidad) y los instantes de *onset* y *offset* con absoluta fidelidad, todo ello preservando intacta la experiencia táctil del intérprete [4].



Figura 3.1: Sistema de sensores de un Yamaha Disklavier, que registra en tiempo real el movimiento e intensidad de cada tecla y pedal durante la interpretación [4].

La alineación resultante entre la señal de audio y las etiquetas MIDI en MAESTRO es de aproximadamente ± 3 ms, muy inferior al umbral de percepción temporal humano (estimado en 20 ms), lo que hace que las etiquetas se consideren alineadas a nivel de fonación. Esta precisión es fundamental para entrenar modelos de transcripción: cualquier desalineación sistemática entre el onset de una nota en el audio y su etiqueta MIDI correspondiente introduce ruido en los gradientes durante el entrenamiento y degrada el rendimiento de los modelos.

3.3 Adquisición y anotación

En primer lugar, el audio se graba en calidad CD o superior, con una frecuencia de muestreo de 44,1 a 48 kHz y una profundidad de bit de 16 bits PCM[1] estéreo. Esta calidad es suficiente para representar con fidelidad toda la gama de frecuencias audibles

[1] PCM (*Pulse Code Modulation*) es una técnica de representación digital de señales analógicas basada en el muestreo y la cuantización, ampliamente utilizada en sistemas de audio sin compresión.

relevantes para el piano (entre 28 Hz y 4 kHz aproximadamente para los fundamentales, con armónicos que se extienden hasta 16-20 kHz). Las grabaciones se almacenan sin compresión con pérdida, lo que preserva toda la información espectral original.

De forma simultánea a la captura del audio, el sistema MIDI del Disklavier captura automáticamente los siguientes eventos para cada interpretación:

- Velocidades de pulsación de teclas (*key strike velocities*, canal MIDI, nota, velocidad 0-127): codifican la dinámica de cada nota individual.
- Posición del pedal *sustain* (Control Change CC64): el pedal más utilizado en el piano, que prolonga el sonido de una tecla.
- Posición del pedal *sostenuto* (Control Change CC66): permite mantener seleccionadas únicamente las notas que están sonando en el momento de su activación.
- Posición del pedal *una corda* (Control Change CC67): pedal izquierdo, que reduce el volumen y modifica el timbre.

La inclusión de los tres pedales hace a MAESTRO uno de los pocos *datasets* de transcripción de piano con **anotación de pedal completa**, lo que resulta especialmente relevante para sistemas que aspiran a una transcripción expresiva o a la síntesis de audio realista a partir de MIDI.

Cada grabación se segmenta automáticamente en piezas musicales individuales y se anota con los siguientes metadatos:

- Compositor canónico (nombre normalizado).
- Título de la obra.
- Año de la grabación.
- Partición sugerida (train/validation/test).

Estos metadatos permiten analizar el repertorio, controlar posibles solapamientos entre particiones y reproducir de forma precisa los protocolos de evaluación propuestos por los autores [3].

3.4 Características del *dataset*

MAESTRO V3.0.0 está formado por un total de 1 276 grabaciones de interpretaciones de piano, que suman 198,7 horas de audio y más de 7,4 millones de eventos MIDI [3]. El repertorio incluye obras de 60 compositores distintos que se asocian principalmente a la música clásica occidental entre los siglos XVII y XX.

Compositor	Grabaciones	Obras únicas	Duración (h)
Frédéric Chopin	201	107	26,22
Franz Schubert	186	96	35,82
Ludwig van Beethoven	146	95	26,92
Johann Sebastian Bach	145	93	13,77
Franz Liszt	131	91	20,27
Sergei Rachmaninoff	59	50	6,66
Robert Schumann	49	31	15,72
Claude Debussy	45	41	5,97
Joseph Haydn	40	29	4,72
Wolfgang Amadeus Mozart	38	35	5,60
Alexander Scriabin	35	26	5,35
Domenico Scarlatti	31	29	1,71
Felix Mendelssohn	28	8	4,85
Johannes Brahms	26	23	8,06
Franz Schubert / Franz Liszt	10	7	0,69
Isaac Albéniz	7	6	0,87
Modest Mussorgsky	6	2	3,17
Pyotr Ilyich Tchaikovsky / Mikhail Pletnev	6	6	0,80
Pyotr Ilyich Tchaikovsky	6	6	0,71
Muzio Clementi	6	6	0,87

Tabla 3.1: Los 20 compositores con mayor número de grabaciones en MAESTRO V3.0.0 [3].

La Tabla 3.1 muestra la distribución típica de competiciones internacionales de piano, que tienden a centrarse en el período Romántico y en los grandes compositores del clasicismo tardío.

Respecto a las particiones, estas están predefinidas y publicadas con el *dataset*, lo que garantiza la reproducibilidad de los experimentos y permite la comparación directa de distintos modelos. Esto es fundamental en un campo como AMT, en el que pequeñas diferencias en la partición de datos pueden producir variaciones significativas en los resultados reportados. En la Tabla 3.2 se detallan las estadísticas principales por partición.

Partición	Grabaciones	Duración (h)	Tamaño (GB)	Notas (M)
Train	962	159,2	96,3	5,66
Validation	137	19,4	11,8	0,64
Test	177	20,0	12,1	0,74
Total	**1,276**	**198,7**	**120,2**	**7,40**

Tabla 3.2: Estadísticas de MAESTRO V3.0.0 desglosadas por partición.

La política de particionado distingue el *dataset* de otras colecciones de audio-MIDI. El criterio establecido por los autores es el siguiente: una misma composición no puede aparecer simultáneamente en más de una partición, aunque haya sido interpretada por distintos pianistas en periodos diferentes [3].

Esta decisión de diseño es crucial para garantizar la validez de los experimentos. Si múltiples interpretaciones de una misma obra aparecieran en entrenamiento y test, un modelo podría aprender características tímbricas o estructurales específicas de esa obra y obtener resultados artificialmente buenos en la evaluación sin haber generalizado correctamente. De esta forma, los modelos se evalúan sobre obras no vistas en entrenamiento, lo que constituye una medida mucho más exigente y realista de rendimiento.

3.5 Evolución del *dataset*

MAESTRO ya va por su tercera versión desde su publicación original en 2018. En la Tabla 3.3 podemos ver características y principales cambios de cada versión.

Versión	Año	Grabaciones	Horas	Notas (M)	Principal novedad
V1.0.0	2018	1,184	172,3	6,18	Publicación original [3]; datos 2004–2016.
V2.0.0	2019	1,282	201,2	7,13	Añade edición 2018 [3]; incorpora pedales *sostenuto* y *una corda*.
V3.0.0	2020	1,276	198,7	7,04	Elimina 6 grabaciones con acompañamiento orquestal [3].

Tabla 3.3: Comparativa de las tres versiones publicadas de MAESTRO.

La transición de V2.0.0 a V3.0.0 merece especial atención. Contrariamente a lo que podría esperarse, la versión más reciente no amplía el *dataset*, sino que realiza una corrección de calidad eliminando 6 grabaciones que no eran de piano solo. La presencia de estas grabaciones en versiones anteriores constituía una inconsistencia que podía afectar negativamente al entrenamiento de modelos especializados en piano solo. Esta corrección hace que las particiones de V3.0.0 no sean compatibles con las de V2.0.0, por lo que todos los experimentos de este trabajo se realizan íntegramente sobre V3.0.0 para garantizar la comparabilidad con los resultados publicados recientemente.

3.6 Distribución y licencia

MAESTRO se distribuye a través de Google Magenta en tres modalidades de descarga [3]:

- **Audio + MIDI** (120,2 GB sin comprimir): contiene los archivos WAV y MIDI completos, junto con los metadatos en formato CSV y JSON.
- **Solo MIDI** ($\sim$ 57 MB): para tareas que no necesiten el audio.
- **Comprimido con pérdida** (MP3/FLAC): versión reducida para almacenamiento limitado.

El *dataset* se distribuye con licencia *Creative Commons Attribution Non-Commercial Share Alike 4.0 International* (CC BY-NC-SA 4.0), que implica:

- Atribución obligatoria: cualquier uso debe citar la fuente original.
- Prohibido su uso con fines comerciales.
- Las obras derivadas deben distribuirse bajo la misma licencia.

Esta licencia es compatible con el uso académico en el contexto de un Trabajo de Fin de Grado.

3.7 Limitaciones

A pesar de sus fortalezas, MAESTRO presenta una serie de limitaciones que debemos tener en cuenta al interpretar los resultados de los sistemas entrenados sobre él.

Sesgo de repertorio. El repertorio es casi exclusivamente música clásica occidental de los siglos XVII-XX. Los modelos entrenados con MAESTRO no generalizan bien a géneros como el jazz, el pop o la música contemporánea con técnicas extendidas. Esto se debe a que las distribuciones de patrones armónicos, rítmicos y de velocidad son muy diferentes a las de la música clásica.

Sesgo de instrumento. Todas las grabaciones se realizaron sobre pianos Yamaha Disklavier del mismo modelo y en condiciones acústicas muy similares. Es por esto que los modelos entrenados con MAESTRO muestran una degradación significativa de rendimiento cuando se evalúan sobre grabaciones de pianos distintos en entornos con reverberación variable. Esto es especialmente relevante para aplicaciones del mundo real, donde el instrumento y la acústica son variables.

Ausencia de anotación de partitura. MAESTRO proporciona MIDI, pero no partitura en formato estándar como MusicXML o MEI [3]. Para tareas de transcripción audio-a-partitura (*Audio-to-Score*), como parte del flujo de las tareas de AMT, es necesario complementar MAESTRO con *datasets* que incluyan partituras anotadas, como ASAP (*Aligned Scores and Performances*).

3.8 Relevancia para este trabajo

MAESTRO V3.0.0 es el *dataset* principal empleado para el entrenamiento y evaluación de los modelos de transcripción automática de piano desarrollados en este trabajo.

Los modelos de referencia del estado del arte, incluyendo los modelos empleados en este trabajo, reportan sus resultados sobre las particiones fijas de MAESTRO, lo que hace directamente comparables los experimentos aquí presentados con los resultados publicados en la literatura.

Su volumen de casi 200 horas con más de **7 millones de eventos MIDI** proporciona datos suficientes para el entrenamiento de arquitecturas basadas en *transformers* sin riesgo de sobreajuste, siempre y cuando se apliquen las técnicas de regularización adecuadas. Todos estos eventos tienen una precisión de alineación de ± 3 ms, garantizando la calidad de las etiquetas *onset*, *offset* y velocidad, que son las variables objetivo de los modelos de transcripción empleados.

Finalmente, las particiones fijas y predefinidas garantizan la reproducibilidad de los experimentos.

3.9 Evaluación estándar

La evaluación cuantitativa de los sistemas de transcripción automática de piano entrenados sobre MAESTRO sigue un protocolo consolidado en la comunidad, implementado mediante la librería `mir_eval` [7]. Este protocolo es el empleado de forma invariante por todos los modelos del estado del arte que reportan resultados sobre este *dataset*, incluyendo los modelos reproducidos en este trabajo, MFT-FiLM y Transkun. Es por esto que en esta sección veremos cómo se evalúan ambos modelos.

Antes de aplicar las métricas, la salida de cada modelo, ya sea en forma de *piano roll* (MFT-FiLM) o en forma de intervalos decodificados (Transkun) se convierte en un conjunto de eventos MIDI discretos de la forma $\{(\text{tono}, t_{\text{onset}}, t_{\text{offset}}, v)\}$, donde el tono es un valor entero en $[21, 108]$ (las 88 teclas del piano estándar), los tiempos se expresan en segundos y la velocidad $v \in [0, 127]$ es un entero MIDI. La evaluación opera directamente sobre estos eventos discretos extraídos del MIDI de salida, comparándolos con

los eventos del MIDI de *ground truth*, sin acceder en ningún momento a la representación interna del modelo, de forma que se garantiza la independencia de las métricas respecto a la arquitectura empleada.

Las métricas previamente mencionadas se organizan en dos categorías en función del nivel de abstracción al que operan: nivel de trama y nivel de nota.

Las **métricas a nivel de trama** (*frame-level*) cuantifican la capacidad del modelo para estimar qué teclas están activas en cada instante temporal, con independencia de cómo se delimiten los eventos. Para ello, tanto la predicción como el *ground truth* se convierten en matrices binarias de dimensiones $T \times 88$, con resolución temporal infinitesimal (equivalente a un piano roll continuo), comparando el estado de activación de cada tecla en cada instante.

Se calculan la **precisión** (P), la **exhaustividad** (R, *Recall*) y la **puntuación** F_1:

$$P = \frac{VP}{VP + FP} \qquad R = \frac{VP}{VP + FN} \qquad F_1 = \frac{2 \cdot P \cdot R}{P + R} \tag{3.1}$$

donde VP son los verdaderos positivos (instantes y teclas correctamente activadas), FP los falsos positivos (activaciones predichas incorrectamente) y FN los falsos negativos (activaciones del *ground truth* no detectadas). Estas métricas de trama son las más directamente relacionadas con la audición subjetiva de la transcripción, ya que capturan el porcentaje de tiempo en que la armonía reconstruida es correcta. En términos musicales, una precisión alta indica que el modelo casi no introduce notas espurias que no aparecen en la grabación original, mientras que una exhaustividad alta implica que rara vez se olvida de activar notas que sí están presentes en la interpretación.

Por otro lado, las **métricas a nivel de nota** (*note-level*) operan sobre los eventos discretos y evalúan la capacidad del modelo para segmentar correctamente los límites temporales de las notas. Hay tres variantes de complejidad creciente:

- **Nota con *onset*** (*Note Onset*): una nota predicha se considera un verdadero positivo si y solo si su tono es correcto y su instante de inicio está dentro de una tolerancia de ± 50 ms respecto al *onset* de *ground truth* correspondiente.
- **Nota con *onset* y *offset*** (*Note w/ Offset*): además de las condiciones anteriores, el instante de finalización de la nota predicha debe encontrarse dentro de una tolerancia igual al máximo entre 50 ms y el 20 % de la duración de la nota de referencia. Esta condición menos restrictiva para el *offset* refleja la mayor dificultad de detección del final de una nota, especialmente en presencia de pedal de *sustain*, cuyo efecto prolonga la resonancia del instrumento más allá del instante de liberación de la tecla.
- **Nota con *onset*, *offset* y velocidad** (*Note w/ Offset & Velocity*): añade una condición sobre la estimación de la dinámica. La velocidad predicha debe satisfacer $|v_{\text{pred}} - v_{\text{ref}}| \leq 0{,}1$ tras escalar ambos valores al intervalo $[0, 1]$, lo que equivale a una tolerancia absoluta de aproximadamente 12,7 unidades MIDI. Esta es la métrica más exigente y la más representativa de la calidad de una transcripción completa de *performance* MIDI.

Desde la perspectiva de un intérprete, estas métricas de nota se aproximan mejor a la noción de "partitura correcta": no basta con que las notas suenen en algún momento, sino que deben empezar y terminar en puntos temporales razonables y con una intensidad coherente con la ejecución original.

Tabla 3.4: Condiciones de aceptación de un evento predicho como verdadero positivo según la variante de métrica de nota empleada.

Métrica	**Tono correcto**	**Tolerancia *onset***	**Condición adicional**
Frame	✓	—	—
Note Onset	✓	$\pm 50\,\text{ms}$	—
Note w/ Offset	✓	$\pm 50\,\text{ms}$	Tolerancia *offset*†
Note w/ Offset & Vel.	✓	$\pm 50\,\text{ms}$	Tolerancia *offset*† + $\lvert\Delta v\rvert \leq 0{,}1$

† máx(50 ms, 20 % de la duración de la nota)

La Tabla 3.4 resume las condiciones de aceptación de cada variante.

Todos los valores de las métricas se **promedian sobre el número de piezas** del conjunto de evaluación, en lugar de calcularse de forma global sobre todas las notas. Este criterio es adoptado de forma uniforme por todos los trabajos de referencia [5, 11, 16].

Los valores de F_1 son los indicadores principales de rendimiento en la literatura de AMT, siendo la ***Note Onset*** F_1 la métrica más citada para comparar modelos, y la ***Note w/ Offset & Velocity*** F_1 la más representativa de la calidad de la transcripción completa.

CAPÍTULO 4
MFT-FiLM

4.1 Introducción

La transcripción de música de piano polifónico mediante redes neuronales se aborda principalmente como un problema de detección multitarea (*Multitask Learning*, o MTL)[8, 5]. En lugar de predecir la representación MIDI en un único paso, el sistema descompone la transcripción en varios objetivos interrelacionados: estimación del tono a nivel de frame (*frame-wise pitch*), la detección del inicio de la nota (*onset*), la detección del final (*offset*), y la estimación de la intensidad o velocidad (*velocity*). Dada la fuerte correlación entre estos eventos, el rendimiento general del sistema mejora significativamente al intercambiar información entre las diferentes tareas.

Sin embargo, las arquitecturas multitarea clásicas presentan limitaciones estructurales [11]. Estrategias como la compartición estricta de parámetros (*hard parameter sharing*) pueden sufrir transferencias negativas debido a conflictos o discrepancias entre los resultados de distintas tareas. Por otro lado, la concatenación o adición directa de las características carece de los pesos adecuados para fusionar dinámicamente las representaciones extraídas por cada rama de la red.

Para abordar estos problemas, en este proyecto, la primera alternativa que se ha implementado es el modelo **MFT-FiLM** (*Multitask Frequency-Time Feature-Wise Linear Modulation*), propuesto por Wang *et al.* [11]. Este sistema introduce un mecanismo de acondicionamiento transversal basado en capas de modulación lineal (*Feature-wise Linear Modulation*, FiLM), diseñadas originalmente para tareas de razonamiento visual, y adaptadas aquí para la fusión de dependencias espectro-temporales en la señal del audio de entrada.

La arquitectura central del MFT-FiLM se basa en dos innovaciones fundamentales:

1. **Módulo FT-FiLM:** un bloque que se encarga de extraer información espectral y temporal de forma jerárquica, usando convoluciones unidimensionales a lo largo del eje de frecuencia y redes neuronales recurrentes (FG-LSTM) para capturar la dependencia temporal de los *frames* de audio [11]. A diferencia de otras arquitecturas, este módulo aplica una transformación afín condicionada para auto-modular las características.

2. **Módulo IFT-FiLM (Inter-Task FiLM):** permite la interacción de información cruzada entre las distintas tareas. Mediante el uso de operaciones de transformación a nivel de tiempo y frecuencia, el sistema modula dinámicamente las características de una rama (por ejemplo *Frame offset*) utilizando como información condicional o auxiliar los parámetros generados por otras ramas (por ejemplo *Frame onset*) [11].

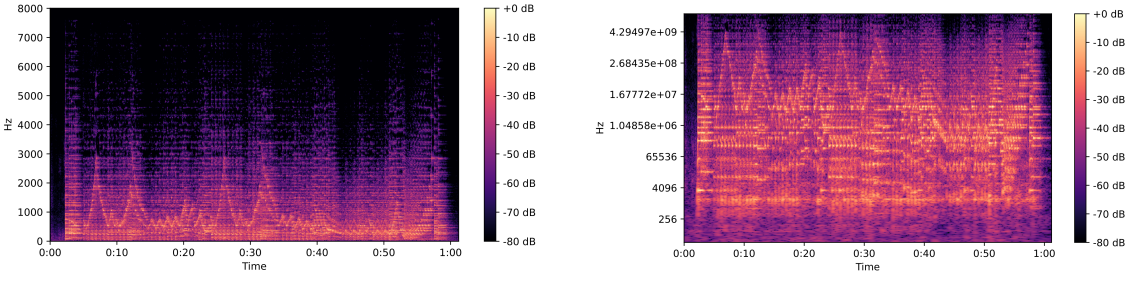
La implementación de este modelo resulta especialmente atractiva para la extracción de *Performance* MIDI debido a su alta eficiencia computacional. A pesar de lograr métricas SOTA, el modelo MFT-FiLM requiere un volumen de parámetros muy pequeño (aproximadamente 1,2 millones), reduciendo la complejidad computacional y costes de entrenamiento sin sacrificar precisión acústica.

En este capítulo, la Sección 4.2 describe la arquitectura del sistema, desde la representación tiempo–frecuencia de entrada hasta la organización de las ramas multitarea y los módulos FiLM. La Sección 4.3 detalla la función de pérdida conjunta empleada y analiza la eficiencia computacional del modelo en términos de número de parámetros y complejidad de cómputo. A continuación, la Sección 4.4 expone el entorno de cómputo y la configuración experimental utilizada para el entrenamiento sobre MAESTRO. En la Sección 4.5 se describe el preprocesamiento del *dataset*, la generación de la caché y el desarrollo práctico del proceso de entrenamiento en el clúster. Finalmente, en la Sección 4.6 se presentan los resultados obtenidos y se discute el rendimiento del modelo frente a trabajos previos del estado del arte.

4.2 Arquitectura del modelo

El primer paso, antes de que la señal de audio entre en la red neuronal, es aplicar una cadena de preprocesamiento. El audio se convierte a mono y se remuestrea a 16 kHz. El oído humano no percibe las frecuencias de forma lineal: la diferencia perceptual entre 100 Hz y 200 Hz es la misma que entre 1000 Hz y 2000 Hz, ya que en ambos casos la segunda es el doble de la primera (diferencia de una octava musical). Las transformaciones tiempo-frecuencia clásicas como la **Transformada de Fourier de tiempo corto** (STFT) distribuyen las bandas de forma **lineal**. Esto implica que las frecuencias graves quedan con poca resolución y las frecuencias agudas con resolución excesiva, lo que no se corresponde con la estructura de la música ni con la percepción auditiva.

En la Figura 4.1 puede apreciarse cómo, para un mismo fragmento de audio (*Prelude op28 no. 16, Chopin*, interpretación original) que presenta una distribución bastante uniforme sobre el piano con presencia en los registros grave y agudo, la STFT lineal agrupa la energía armónica en muy pocas bandas graves mientras que la CQT asigna un número constante de *bins* por octava, distribuyendo la información de forma mucho más uniforme a lo largo del rango del piano.



(a) STFT con escala lineal de frecuencia.

(b) CQT con escala logarítmica tipo piano.

Figura 4.1: Comparación entre un espectrograma STFT con eje de frecuencia lineal y un espectrograma CQT con eje logarítmico. Imágenes generadas a partir de un mismo fragmento de audio.

Es por esto que se usa la **Transformada Q Constante** (*Constant-Q Transform*, CQT), que distribuye las bandas de frecuencia en escala logarítmica, alineándose con la percepción musical y con la disposición de las teclas en el piano [11]. La CQT presenta los siguientes parámetros de configuración:

- Frecuencia mínima de 27,5 Hz correspondiente a la tecla A0 (la tecla más grave de un piano estándar).
- 48 *bins* por octava, con un total de 352.
- Ventana de *Hanning* y salto de trama (*hop length*) de 20 ms.

El espectrograma CQT de dimensiones $(T \times 352)$, donde T es el número de *frames*, se remodela a (C_0, T, F) con $C_0 = 4$ y $F = 88$. De esta forma, los 352 *bins* se distribuyen en cuatro grupos de 88 frecuencias, en correspondencia biunívoca con las 88 teclas del piano [11]. Esta representación constituye la entrada de la capa lineal inicial del modelo, que se somete a una serie de módulos FiLM que veremos a continuación.

A modo de esquema general, en la Figura 4.2 se presenta un diagrama de la arquitectura del modelo MFT-FiLM que servirá de guía para los diguientes apartados. Este muestra el flujo de la información desde la entrada a la salida y la interacción que existe entre las tres ramas principales.

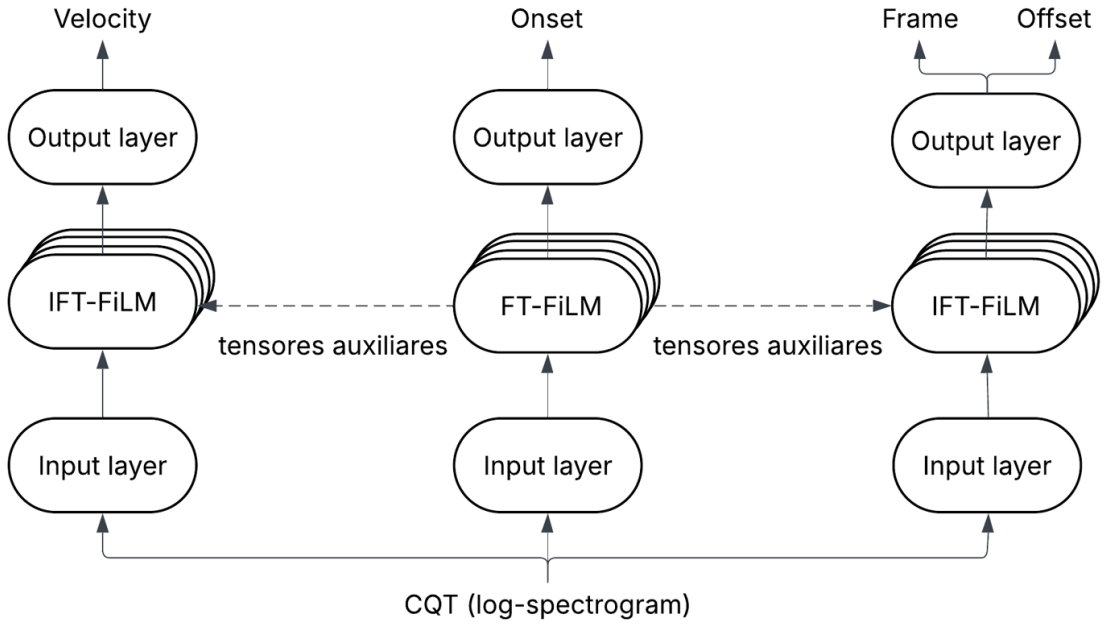


Figura 4.2: Esquema general del modelo MFT-FiLM para la transcripción multitarea de notas de piano, con las tres ramas de *onset*, *velocity* y *frame+offset*, y el flujo de información a través de los bloques FT-FiLM e IFT-FiLM. Ilustración adaptada de [11].

El mecanismo *Feature-Wise Linear Modulation* (FiLM) fue propuesto inicialmente por Perez *et al.* (2018) [9] para tareas de razonamiento visual. La idea central consiste en condicionar las activaciones de una red neuronal mediante una transformación afín por cada elemento, usando información de contexto generada por un módulo externo denominado *FiLM Generator*. Definimos formalmente la operación como:

$$\text{FiLM}(x, \gamma, \beta) = \gamma \odot x + \beta \tag{4.1}$$

donde x es el mapa de características de entrada, γ y β son los parámetros de escala y desplazamiento generados dinámicamente, y $\odot$ es el producto de Hadamard (multiplicación elemento a elemento). El principal beneficio de este mecanismo respecto a la suma o concatenación directa de características es que γ y β se adaptan en función del contexto

en cada paso de inferencia, permitiendo al modelo ponderar y suprimir canales de forma dinámica en lugar de aplicar pesos fijos.

El módulo **FT-FiLM** (*Frequency-Time FiLM*), implementado en la rama central (Figura 4.2), adapta el mecanismo descrito en la ecuación (4.1) para extraer dependencias espectrales y temporales de forma jerárquica a partir del espectrograma CQT. Su operación se define como:

$$\text{FT-FiLM}(x) = f_{FT}(x) \odot x + h_{FT}(x) \tag{4.2}$$

donde f_{FT} y h_{FT} son generadores de los parámetros de modulación γ y β respectivamente, extrayendo la información espectro-temporal en tres etapas secuenciales:

Convolución en el eje de frecuencia. El mapa de características de entrada, de dimensiones (C, T, F), se convoluciona con un kernel 1D de tamaño $2F - 1 = 175$ sobre el eje de frecuencia. Este kernel se diseña con una longitud igual al eje completo para capturar simultáneamente las relaciones armónicas entre todos los *bins* de frecuencia del piano, algo que las convoluciones de kernel reducido no pueden lograr [11].

Transformación de canales. Los canales se expanden de C a $6C$ mediante una convolución 1D con kernel de tamaño 1, y el resultado se divide en dos tensores z y z', cada uno de C canales, que servirán respectivamente como parámetros de escala y desplazamiento de la modulación afín.

Modelado temporal con FG-LSTM. Los tensores z y z' se procesan con dos redes recurrentes **FG-LSTM** (*Frequency-Grouped LSTM*) bidireccionales independientes, cada una con 32 unidades ocultas, que capturan las dependencias temporales entre *frames* consecutivos. El uso de LSTM bidireccional incorpora tanto el contexto pasado como el futuro en cada instante de la secuencia.

Tras estas tres etapas, el mapa de entrada original x se modula aplicando la transformación afín de la ecuación (4.2), con una capa de *dropout* de tasa 0,1 antes de la modulación final. El resultado es una representación auto-condicionada que integra información espectral y temporal de forma jerárquica y separada, lo que permite mantener un número de parámetros reducido.

Con el objetivo de permitir la interacción entre ramas de distintas tareas, se crea el módulo IFT-FiLM (*Inter-Task FT-FiLM*), que extiende el FT-FiLM. Lo formulamos como:

$$\text{IFT-FiLM}(x,\, x') = f_{IFT}(x,\, x') \odot x + h_{IFT}(x,\, x') \tag{4.3}$$

donde x es el mapa de características de la rama principal y x' es la información auxiliar proveniente de otra rama. La diferencia estructural respecto al FT-FiLM reside en el paso de transformación de canales: los tensores z y z' generados por la rama auxiliar se concatenan a los de la rama principal a lo largo del eje de canales antes de reducirlos de vuelta a C mediante una convolución 1D con kernel de tamaño 1.

Un detalle de implementación relevante es que los tensores auxiliares se **desconectan del grafo de cómputo** (`detach`) antes de ser incorporados [11]. Esta operación impide que el gradiente fluya de vuelta hacia la rama auxiliar a través de esta conexión, de forma que evitamos inestabilidades en la optimización cuando dos ramas se condicionan mutuamente. El módulo IFT-FiLM puede interpretarse como una red neuronal dinámica: los parámetros de modulación no son fijos, sino que se generan en cada paso en función

de las entradas de ambas ramas, permitiendo que la red adapte su comportamiento de acuerdo con el contexto conjunto de las tareas involucradas.

Ya con los módulos FiLM definidos, se puede describir el sistema completo de transcripción. Este está compuesto por **tres ramas paralelas**: *onset*, *velocity* y *frame+offset*. Cada rama apila cuatro bloques FT-FiLM o IFT-FiLM en cascada [11], y el flujo de información cruzada entre ramas sigue la siguiente lógica:

- La **rama *onset*** emplea bloques FT-FiLM puros (auto-condicionamiento), ya que detectar el comienzo de una nota es la subtarea más discriminativa del sistema.
- En cada uno de los cuatro niveles, los tensores z y z' generados por la rama *onset* se desconectan y se reenvían a las ramas *velocity* y *frame*, que los incorporan mediante bloques IFT-FiLM.
- De esta forma, el diseño produce **cuatro niveles de interacción cruzada**, desde representaciones de bajo nivel espectral hasta semánticas de alto nivel, enriqueciendo progresivamente las representaciones de cada subtarea.

El flujo unidireccional de información (onset $\rightarrow$ velocity, onset $\rightarrow$ frame) está justificado por el hecho de que la señal de *frame* combina información de notas sostenidas y de finales, lo que podría interferir de forma negativa en la detección precisa de inicios si se utilizara como entrada de la rama *onset* [11]. Las dimensiones de los canales son $C = 64$ en las ramas *onset* y *velocity*, y $C = 32$ en la rama *frame+offset*, lo que fomenta la minimización de los parámetros del modelo.

Cada rama dispone de una capa de salida que ajusta el número de canales al objetivo de la subtarea correspondiente. La capa sigue el esquema: expansión a 2C canales, seguida de una capa de *dropout* con tasa 0,1, activación GELU (que actúa como una versión suavizada de una ReLU), y compresión al número de canales objetivo [11]. La Tabla 4.1 resume las configuraciones de salida de cada rama.

Tabla 4.1: Configuración de las capas de salida de cada rama del sistema MFT-FiLM.

Rama	Canales de salida	Activación	Dimensión del piano roll
Onset	1	Sigmoide	$T \times 88$
Frame+Offset	2	Sigmoide	$T \times 88\,(\times 2)$
Velocity	128	—	$T \times 88 \times 128$

Por último, cabe mencionar que durante la inferencia se aplica una estrategia de **medio paso** (*half stride*), que procesa el audio en ventanas con un solapamiento del 50 % para mitigar los artefactos en los bordes de cada bloque. El algoritmo de decodificación heurístico convierte los piano rolls de probabilidad en eventos de nota discretos (tono, inicio, final, velocidad), con una tolerancia de 50 *ms* en los inicios; y del máximo entre 50 *ms* y el 20 % de la duración en los finales.

4.3 Función de pérdida y eficiencia

La función de pérdida del sistema es la suma de las pérdidas individuales de cada subtarea:

$$\ell = \ell_{\text{onset}} + \ell_{\text{frame}} + \ell_{\text{offset}} + \ell_{\text{velocity}} \tag{4.4}$$

Las pérdidas de *onset*, *frame* y *offset* se calculan mediante **entropía cruzada binaria** (BCE), apropiada para clasificación binaria píxel a píxel del piano roll:

$$\ell_{\text{onset}} = \sum_{k=1}^{K} \sum_{t=1}^{T} \ell_{\text{BCE}}(I_{\text{onset}}(t,k),\ P_{\text{onset}}(t,k)) \tag{4.5}$$

y análogamente para ℓ_{frame} y ℓ_{offset}. La pérdida de velocidad se calcula mediante **entropía cruzada categórica** (CCE) sobre 128 clases, dado que la velocidad MIDI es un valor entero en el intervalo $[0, 127]$:

$$\ell_{\text{velocity}} = \sum_{k=1}^{K} \sum_{t=1}^{T} \ell_{\text{CCE}}\big(I_{\text{velocity}}(t,k),\ P_{\text{velocity}}(t,k)\big) \tag{4.6}$$

donde I y P representan los valores de *ground truth* y predicción respectivamente, $K = 88$ es el número de teclas del piano y T el número de *frames* [11]. La ausencia de factores de ponderación entre las cuatro pérdidas es una elección de diseño deliberada por los autores, cuya validez queda respaldada por el estudio de ablación presentado en la sección de resultados [11].

Una de las aportaciones más destacadas del modelo es su reducido tamaño. Con tan solo 1,2 **millones de parámetros**, el MFT-FiLM se sitúa como uno de los modelos de transcripción de piano más eficientes del estado del arte. En términos de operaciones de multiplicación-acumulación (MACs), calculadas para una salida de 128 *frames* con la biblioteca `ptflops`, el MFT-FiLM registra una complejidad notablemente inferior a la de otros modelos como el hFT-Transformer o el SemiCRF V2 [11, 13, 16].

Esta eficiencia no sacrifica capacidad expresiva gracias al diseño jerárquico del generador FiLM, que desacopla el procesamiento en los ejes de frecuencia, canal y tiempo en lugar de operarlos conjuntamente. La separación de estas dimensiones permite reutilizar módulos ligeros (convoluciones 1D y LSTMs de baja dimensionalidad) en lugar de recurrir a mecanismos de atención de complejidad cuadrática, sin renunciar a la capacidad de capturar dependencias espectrales y temporales de largo alcance.

4.4 Configuración del experimento

El entrenamiento del modelo MFT-FiLM se llevó a cabo en el clúster de cómputo de alto rendimiento **tensor**, que pertenece al Departamento de Sistemas Informáticos y Computación (DSIC) de la Universidad Politécnica de Valencia. Este sistema está formado por un único nodo equipado con 503 *GB* **de memoria RAM** y cuatro unidades de procesamiento gráfico **NVIDIA L40S** con 48 *GB* de memoria GDDR6 ECC cada una. Las GPUs L40S son tarjetas de la familia *Ada Lovelace*, orientadas específicamente a cargas de trabajo de inteligencia artificial sostenidas en entornos de servidor. A diferencia de las tarjetas de consumo de la misma generación, estas incorporan memoria ECC (*Error-Correcting Code*), que detecta y corrige errores de bit durante el cómputo, algo que resulta realmente útil en trabajos de larga duración como el entrenamiento de redes neuronales.

El hardware empleado en el artículo original consistía en dos tarjetas NVIDIA GeForce RTX 4090 de 24 *GB* de VRAM cada una [11]. La L40S cuenta individualmente con 48 *GB*, lo que permitió alojar el modelo con lotes de tamaño deseado sin problemas.

El acceso se realiza mediante SSH a través del nodo de entrada `tensor.dsic.upv.es`, autenticado con credenciales institucionales. El sistema de gestión de trabajos es **SLURM** (*Simple Linux Utility for Resource Management*), que organiza la ejecución de trabajos mediante dos colas principales: `docencia`, para trabajos de duración inferior a 24 horas; y

`long`, para trabajos de hasta 72 horas. Dada una duración de entrenamiento estimada por los autores de 43 horas, fue necesario usar la cola `long`.

Todas las dependencias se encapsularon en un entorno virtual de Python para garantizar la reproducibilidad del experimento y para aislar las versiones de las librerías del sistema global del clúster. El uso de entornos virtuales es una práctica estándar en proyectos de aprendizaje automático, ya que distintos proyectos pueden requerir versiones incompatibles de las mismas librerías. El proceso de creación fue el siguiente:

1. Creación del entorno virtual con el intérprete Python 3.12 disponible en el clúster:

```
mkdir -p ~/venv
python3 -m venv ~/venv/MFT-FiLM
source ~/venv/MFT-FiLM/bin/activate
```

2. Instalación de PyTorch con soporte para CUDA 12.6, versión de controladores disponible en `tensor`:

```
pip install torch torchvision torchaudio \
    --index-url https://download.pytorch.org/whl/cu126
```

3. Instalación del resto de dependencias del repositorio:

```
pip install prettymidi mireval librosa==0.9.2 \
    numpy scipy joblib tensorboard
```

Después de preparar el entorno, se clonó el repositorio público desde el GitHub oficial [11], y se creó un enlace simbólico para trabajar con MAESTRO previamente descargado. Durante la realización de pruebas para comprobar el funcionamiento del código original encontramos ciertas dificultades. En los siguientes apartados veremos en detalle los problemas técnicos más relevantes y las soluciones adoptadas para resolverlos.

Problema 1: Obsolescencia de dependencias. La reproducibilidad en aprendizaje automático aplicado a la recuperación de información musical es altamente sensible a las versiones de las librerías de procesamiento de señal. Para garantizar que las matrices de los espectrogramas CQT generados coincidieran con las reportadas en el artículo original, fue estrictamente necesario usar la versión **0.9.2** de la librería `librosa`. Las versiones más recientes (a partir de la 0.10) introdujeron modificaciones en el cómputo del *padding* interno del CQT, lo que produce una diferencia de uno o dos *frames* entre el tensor del audio y el tensor de etiquetas MIDI generado a partir del mismo archivo. Dado que el código asume una correspondencia exacta entre ambas dimensiones temporales, este desajuste provocaba un error fatal durante la inicialización del *DataLoader*.

El clúster `tensor` opera con un intérprete **Python 3.12**. A partir de esta versión, las herramientas de construcción base del ecosistema Python (`setuptools` v70+) eliminaron por completo el módulo `pkg_resources`, considerado obsoleto. Sin embargo, la versión 0.9.2 de `librosa` posee una dependencia rígida en dicho módulo para localizar e instanciar sus archivos internos de configuración. Esto generó un bloqueo de entorno: el modelo requería la librería heredada para mantener la integridad dimensional de los datos, pero el intérprete moderno de Python impedía su carga, abortando el proceso de entrenamiento antes incluso de la fase de carga de datos.

Para solventarlo sin alterar la versión global de Python de `tensor` y sin comprometer la versión de `librosa`, se aplicó una degradación forzada de las herramientas de construcción del entorno virtual:

```
pip install "setuptools==69.5.1" --force-reinstall
```

La versión 69.5.1 es la última versión de transición que conserva el módulo necesario `pkg_resources` íntegro en el espacio de usuario. Esta acción creó un puente de compatibilidad que permitió ejecutar código de procesamiento de señal de generación anterior sobre un intérprete Python de última generación, sin necesidad de modificar ningún archivo del código fuente del repositorio ni alterar el entorno global del sistema.

Problema 2: Incompatibilidad de la API de PyTorch y corrupción de la caché. Como veremos posteriormente, el entrenamiento requiere una fase previa de **preprocesamiento masivo paralelo** para extraer todos los espectrogramas y los *piano rolls* de las etiquetas MIDI, almacenándolos en archivos binarios `.pth` en disco (caché). Tras finalizar este proceso, la clase constructora del *dataset* (`CacheDataset`) debía cargar dichos tensores en memoria para ejecutar el bucle de entrenamiento del modelo.

Al iniciarse la fase de entrenamiento, el código produjo una excepción crítica:

```
AssertionError: Size mismatch between tensors
```

En tareas de recuperación de información musical, este tipo de error suele indicar un desajuste en el número de *frames* entre el espectrograma de audio y la correspondiente matriz de etiquetas (por ejemplo, un audio que genera 1.000 *frames* y cuyo *piano roll* genera 1.001 debido a redondeos en aritmética de coma flotante). Sin embargo, tras analizar en profundidad el código fuente de los autores, se identificó que la causa era un **defecto de diseño arquitectónico** expuesto por la actualización de PyTorch.

Los autores definieron la clase `CacheDataset` heredando de `TensorDataset`. No obstante, en su método inicializador, invocaban a la clase padre sin pasar ningún tensor (`super(CacheDataset, self).__init__()`), ya que gestionaban la carga en memoria de forma personalizada. Las versiones antiguas de PyTorch (serie 1.x) aceptaban esta inicialización vacía de forma silenciosa. Las versiones modernas (serie 2.x, empleadas en `tensor`) incorporan validaciones internas que asumen que la ausencia de tensores en la inicialización implica un desajuste dimensional, lanzando la excepción de forma sistemática. Adicionalmente, las interrupciones abruptas producidas durante los primeros intentos corrompieron parcialmente los archivos `.pth` de caché generados, dejando el sistema en un estado irrecuperable que exigió intervención manual.

Para solventar los problemas, por un lado se modificó la declaración de herencia de la clase `CacheDataset` en el archivo `data/dataload.py`, sustituyendo la herencia de `TensorDataset` por la clase base estándar `torch.utils.data.Dataset`:

```
# Antes:
class CacheDataset(TensorDataset):
    def __init__(self, ...):
        super(CacheDataset, self).__init__()
        ...

# Después:
class CacheDataset(torch.utils.data.Dataset):
    def __init__(self, ...):
        super(CacheDataset, self).__init__()
        ...
```

Al heredar de la clase base genérica, se evitó la validación estricta de tensores en la inicialización, delegando la responsabilidad de la gestión dimensional a los métodos `__getitem__` y `__len__`, que ya estaban correctamente implementados por los autores. Por otro lado, se eliminó completamente el árbol de directorios de caché corrupta y se regeneraron los archivos desde cero.

Tras solucionar todos los problemas e incompatibilidades, se procedió con la configuración de parámetros. El archivo central de configuración es `constant_macro.py`, que controla todos los hiperparámetros y rutas del sistema. Para replicar las condiciones descritas en el artículo original, se ajustaron los parámetros que se muestran en la Tabla 4.2 [11].

Tabla 4.2: Parámetros de configuración establecidos en `constant_macro.py` para el entrenamiento de MFT-FILM.

Parámetro	Valor	Descripción
`EPOCH_RANGE`	60	Número total de épocas de entrenamiento
`BATCH_SIZE`	10	Número de muestras por lote
`LEARNING_RATE`	0,0005	Tasa de aprendizaje del optimizador
`DATASET_NAME`	`maestro-v3.0.0`	Versión del *dataset*
`DATASET_PATH`	ruta absoluta	Directorio raíz de los datos
`MODEL_PATH`	ruta absoluta	Directorio de salida del modelo
`MODEL_NAME`	`MTFiLM`	Identificador del modelo
`SCHEDULE_IDX`	`2x`	Objetivo: transcripción de notas

A continuación se analiza el significado y se justifica la elección de los parámetros más relevantes.

Optimizador y tasa de aprendizaje. El optimizador empleado es **Adam** (*Adaptive Moment Estimation*), considerado estándar en el entrenamiento de redes para tareas de procesado de señal. Este optimizador mantiene para cada parámetro del modelo, estimaciones del primer momento (media exponencial de los gradientes pasados, m_t) y del segundo momento (media exponencial de los cuadrados de los gradientes, v_t), actualizando cada parámetro θ según:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \tag{4.7}$$

donde $\hat{m}_t$ y $\hat{v}_t$ son las estimaciones corregidas por sesgo de ambos momentos, α es la tasa de aprendizaje global, y ϵ es un término de estabilidad numérica. La principal ventaja de Adam sobre el descenso de gradiente estocástico clásico (SGD) es que la tasa de aprendizaje efectiva de cada parámetro se adapta automáticamente en función de la magnitud histórica de sus gradientes. Los parámetros que reciben gradientes de alta magnitud con frecuentemente se actualizan menos veces, mientras que aquellos con gradientes escasos o de baja magnitud se actualizan más veces. Este comportamiento es especialmente beneficioso en arquitecturas multitarea como el MFT-FiLM, donde diferentes ramas de la red pueden tener gradientes de magnitud muy dispar.

La tasa de aprendizaje global se fijó en $\alpha = 5 \times 10^{-4}$, valor utilizado por los autores originales [11].

Tamaño de lote. El **tamaño de lote** (*batch size*) de 10 espectrogramas es el valor reportado en el artículo [11], y difiere del valor predeterminado del repositorio (4 muestras). El

batch size es relevante desde dos puntos de vista: la estabilidad del gradiente y el hardware.

Desde el punto de vista de la optimización, valores elevados producen estimaciones del gradiente con menor varianza, lo que se traduce en actualizaciones de parámetros más estables y convergencia más suave. Lotes excesivamente grandes pueden conducir a mínimos agudos con peor capacidad para generalizar. El valor de 10 representa un equilibrio pragmático entre ambos extremos para este dominio.

Desde el punto de vista del hardware, cada muestra de entrenamiento es un tensor de espectrograma de dimensiones $(4, T, 88)$, donde $T = 400$ *frames* (correspondiente a una ventana de 8 segundos con *hop* de 20 ms). Un lote de 10 de estas muestras ocupa aproximadamente 40 GB de la VRAM de la GPU L40S, situando la utilización de memoria cerca del límite físico de 48 GB disponibles. Lotes de mayor tamaño habrían requerido el uso de más gráficas para alojar el modelo durante el entrenamiento.

Épocas y guardado. El número de **épocas** se fijó en 60, conforme a la configuración descrita en el artículo para el entrenamiento sobre MAESTRO [11]. Una época representa una pasada completa del optimizador sobre todos los ejemplos del conjunto de entrenamiento; con 7 186 lotes de 10 muestras, cada época implica 7 186 actualizaciones de los parámetros del modelo. El número de épocas determina la cantidad total de actualizaciones de gradiente y, por tanto, el grado de especialización del modelo sobre el conjunto de entrenamiento.

Para minimizar el riesgo de sobreajuste, se implementó una estrategia de **guardado dinámico por validación**. Al principio (10 épocas), la evaluación del modelo sobre el conjunto de validación se realiza cada 5 épocas. Una vez supera un umbral mínimo de convergencia, la validación se realiza cada época, guardando un *checkpoint* únicamente cuando se observa una mejora respecto al mejor valor histórico de pérdida en validación. De esta forma, el modelo final seleccionado no es necesariamente el de la última época completada, sino el que mejor se comportó en validación.

4.5 Preproceso y entrenamiento

Antes del entrenamiento, se ejecuta una fase de **preprocesamiento y caché** del *dataset*. Durante esta fase se transforma cada audio del conjunto MAESTRO en su espectrograma CQT mediante la librería `librosa`, y los archivos MIDI correspondientes son procesados mediante `pretty_midi` para generar los *piano rolls* de *ground truth*.

Adicionalmente, la duración de cada nota en el *ground truth* se extiende en función de la información de los pedales de *sustain* y *sostenuto*, siguiendo el siguiente criterio: una nota se prolonga hasta que el valor del pedal de sustain en el archivo MIDI cae por debajo de 64, o hasta que la misma tecla es pulsada de nuevo. Este paso es crucial para la fidelidad del *ground truth*, ya que en interpretaciones de piano con pedal, las notas resuenan durante más tiempo del que dura la pulsación física de la tecla; ignorar esta información produciría un *ground truth* sistemáticamente más corto que el audio real.

El proceso se paraleliza mediante la librería `joblib`, y el resultado de la ejecución es un conjunto de archivos binarios que serán reutilizados directamente en todos los entrenamientos subsiguientes, eliminando el coste de reprocesamiento.

Dado que el tiempo estimado de entrenamiento excedía ampliamente el límite de 24 horas de la cola `docencia`, el trabajo se sometió a la cola `long` de SLURM. Se solicitó una

única GPU L40S, ya que, como hemos visto, es suficiente para alojar el modelo con el *batch size* seleccionado.

Durante el entrenamiento se observó un patrón de utilización de la GPU con oscilaciones periódicas de uso entre el 0 % y el 100 %, en lugar de un valor cercano al máximo continuado. Este fenómeno, conocido como *GPU starvation* o cuello de botella de entrada/salida (*I/O bottleneck*), se produce cuando la velocidad de cómputo de la GPU es significativamente superior a la velocidad con que la CPU puede preparar y transferir nuevos lotes de datos a la VRAM.

El mecanismo es el siguiente: una vez que la GPU termina de procesar un lote de 10 espectrogramas (operación que, dada la compacidad del modelo, se completa en pocos segundos), el proceso queda bloqueado a la espera de que la CPU lea el siguiente lote desde los archivos `.pth` en disco y lo transfiera a la VRAM a través del bus PCIe. Durante ese intervalo de espera, la utilización de la GPU cae al 0 %. En la configuración empleada, este fenómeno fue particularmente pronunciado debido a que `tensor` solo tiene un nodo, compartido por todos los usuarios, y no se puede forzar el `DataLoader` para usar continuamente todos los `num_workers` disponibles.

Como consecuencia directa de este cuello de botella, la duración efectiva de cada época fue de aproximadamente **83 minutos**, frente a los 43,5 minutos reportados por los autores originales con dos GPUs RTX 4090. En el *setup* original, la paralelización entre dos GPUs permitía solapar parcialmente la fase de transferencia de datos con el cómputo sobre la segunda tarjeta, mitigando significativamente este efecto y casi duplicando el rendimiento efectivo.

El clúster `tensor` impone una duración máxima de **72 horas** para los trabajos sometidos a la cola `long`. Dado que cada época requirió aproximadamente 83 minutos —prácticamente el doble del tiempo estimado inicialmente—, el trabajo de entrenamiento fue interrumpido por SLURM al alcanzar dicho límite temporal, habiendo completado únicamente **30 de las 60 épocas planificadas**.

A pesar de no haber completado el ciclo de entrenamiento establecido en el artículo, el mecanismo de validación dinámica y guardado automático de *checkpoints* preservó el mejor modelo alcanzado durante las 30 épocas ejecutadas. Este modelo resultó lo suficientemente bueno en la evaluación, obteniendo resultados muy cercanos al estado del arte, que analizaremos en la siguiente sección.

4.6 Resultados

En esta sección se analizan los resultados obtenidos por el modelo MFT-FiLM entrenado sobre MAESTRO v3.0.0. Las métricas empleadas coinciden con el protocolo estándar descrito en la Sección 3.9: se evalúa el rendimiento a nivel de trama y a nivel de nota mediante precisión, *recall* y puntuación F_1, tanto para la activación de teclas (*Frame*) como para las variantes *Note Onset*, *Note w/ Offset* y *Note w/ Offset & Velocity* [11, 16].

Para garantizar la comparabilidad directa con los trabajos originales, las predicciones de MFT-FiLM se umbralizan utilizando los mismos valores que en el repositorio de referencia. En todas las evaluaciones se fijan los umbrales de decisión a $t_{\text{onset}} = 0{,}4$, $t_{\text{frame}} = 0{,}48$ y $t_{\text{offset}} = 0{,}5$ [11].

Las Tablas 4.3, 4.4 y 4.5 muestran una comparación cuantitativa de los resultados obtenidos sobre MAESTRO v3.0.0, incluyendo las métricas de F_1, precisión y *recall*, respectivamente. Se compara el modelo entrenado con otros modelos SOTA.

Tabla 4.3: Resultados de transcripción de notas sobre MAESTRO v3.0.0. Se muestran el número de parámetros y las puntuaciones F_1 a nivel de trama (*Frame*) y de nota (*Note Onset*, *Note w/ Offset*, *Note w/ Offset & Velocity*). Todas las métricas se expresan en porcentaje (%).

Modelo	**Parámetros**	**Frame** F_1	**Onset** F_1	**Onset+Off.** F_1	**Onset+Off.+Vel.** F_1
Transformer [12]	54 M	—	96.01	83.94	82.75
SemiCRF [15]	9 M	90.75	96.11	88.42	87.44
HPPNet [14]	1.2 M	93.15	97.18	83.80	82.24
hFT-Transformer [13]	5.5 M	93.24	97.44	90.53	89.48
HATANet [10]	6.4 M	94.04	97.81	90.57	89.25
SemiCRF V2 [16]	12.9 M	95.35	98.32	93.48	92.94
MFT-FiLM (autores) [11]	1.2 M	94.75	98.39	92.46	91.79
MFT-FiLM (este trabajo)	1.2 M	94.36	98.17	91.60	90.78

Tabla 4.4: Resultados de precisión de transcripción de notas sobre MAESTRO v3.0.0. Se muestran el número de parámetros y las métricas de precisión a nivel de trama (*Frame*) y de nota (*Note Onset*, *Note w/ Offset*, *Note w/ Offset & Velocity*). Todas las métricas se expresan en porcentaje (%).

Modelo	**Parámetros**	**Frame Precisión**	**Onset Precisión**	**Onset+Off. Precisión**	**Onset+Off.+Vel. Precisión**
Transformer [12]	54 M	—	98.61	86.19	84.95
SemiCRF [15]	9 M	93.79	98.69	90.79	89.78
HPPNet [14]	1.2 M	92.79	98.45	84.88	83.29
hFT-Transformer [13]	5.5 M	92.82	99.64	92.52	91.43
HATANet [10]	6.4 M	95.51	99.05	91.70	90.35
SemiCRF V2 [16]	12.9 M	95.75	99.53	94.61	94.07
MFT-FiLM (autores) [11]	1.2 M	95.77	99.33	93.33	92.65
MFT-FiLM (este trabajo)	1.2 M	95.27	99.37	92.70	91.86

Tabla 4.5: Resultados del *recall* en transcripción de notas sobre MAESTRO v3.0.0. Se muestran el número de parámetros y las métricas de *recall* a nivel de trama (*Frame*) y de nota (*Note Onset*, *Note w/ Offset*, *Note w/ Offset & Velocity*). Todas las métricas se expresan en porcentaje (%).

Modelo	**Parámetros**	**Frame Recall**	**Onset Recall**	**Onset+Off. Recall**	**Onset+Off.+Vel. Recall**
Transformer [12]	54 M	—	93.60	81.86	80.70
SemiCRF [15]	9 M	88.36	93.96	86.46	85.51
HPPNet [14]	1.2 M	93.59	95.95	82.76	81.24
hFT-Transformer [13]	5.5 M	93.66	95.44	88.69	87.67
HATANet [10]	6.4 M	92.62	96.61	89.47	88.17
SemiCRF V2 [16]	12.9 M	95.01	97.16	92.39	91.87
MFT-FiLM (autores) [11]	1.2 M	93.84	97.48	91.62	90.95
MFT-FiLM (este trabajo)	1.2 M	93.56	97.03	90.54	89.73

La Tabla 4.3 muestra que el modelo MFT-FiLM entrenado alcanza unas puntuaciones F_1 muy próximas a las reportadas por los autores originales en MAESTRO v3.0.0 [11]. A pesar de haber completado únicamente 30 de las 60 épocas de entrenamiento descritas en el artículo, el F_1 de trama se mantiene por encima del 94 %, mientras que las métricas de nota alcanzan valores superiores al 98 % en *Note Onset* y en torno al 91–91,5 % en *Note w/ Offset* y *Note w/ Offset & Velocity*. Estas diferencias de unas pocas décimas respecto a los resultados publicados pueden atribuirse tanto a la reducción del número de épocas como a las pequeñas divergencias en el entorno de cómputo ya comentadas en la Sección 4.5.

Las Tablas 4.4 y 4.5 descomponen el F_1 en términos de precisión y *recall*. En la reproducción se observa una precisión especialmente alta en la detección de inicios de nota, cercana al 99 %, lo que indica que el modelo apenas introduce notas espurias que no existen en la partitura de referencia. El *recall* en onsets se sitúa ligeramente por debajo, en torno al 97 %, lo que sugiere que los pocos errores restantes se deben principalmente a notas omitidas en pasajes complejos más que a activaciones falsas.

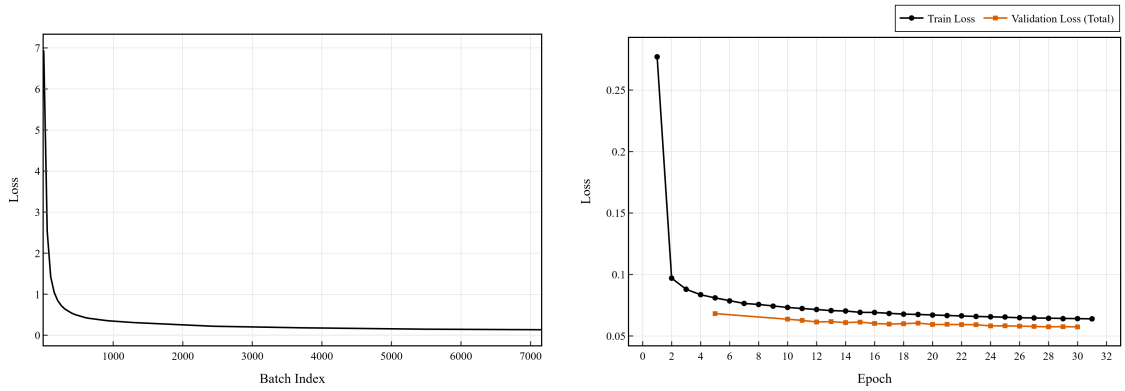
En las métricas que incluyen el *offset* y la velocidad, tanto la precisión como el *recall* descienden de forma moderada pero se mantienen en torno al 90 %, en línea con los re-

sultados de MFT-FiLM reportados por Wang *et al.* [11] y con los mejores modelos del estado del arte. Esto refleja la mayor dificultad de estimar con precisión el final de las notas y su dinámica en presencia del pedal de *sustain*, pero confirma que la arquitectura MFT-FiLM es capaz de modelar de forma consistente la articulación temporal y la intensidad de la interpretación, incluso cuando el entrenamiento se detiene a mitad de las épocas previstas.

Finalmente, la comparación horizontal de las tres tablas evidencia que, con solo 1,2 millones de parámetros, la variante reproducida de MFT-FiLM ofrece un rendimiento muy cercano al de modelos considerablemente más pesados, como hFT-Transformer o las variantes SemiCRF. Desde la perspectiva de la relación entre eficiencia y precisión, este comportamiento respalda la tesis de los autores de que la modulación lineal frecuencia–tiempo permite explotar de forma eficaz la información compartida entre tareas sin necesidad de incrementar el tamaño del modelo.

De forma complementaria a la evaluación del rendimiento final sobre el conjunto de prueba, el análisis de las métricas durante el proceso de entrenamiento resulta fundamental para comprender el comportamiento analítico y la eficiencia de aprendizaje de la arquitectura MFT-FiLM. Con este propósito, se ha monitorizado la evolución de la función de pérdida (*loss*) y de las métricas de validación a lo largo de las 30 épocas completadas.

La Figura 4.3 ilustra la dinámica de minimización del error, presentando de forma paralela la reducción de la pérdida intra-época (evaluada durante los primeros lotes) y la evolución macroscópica a lo largo de la totalidad del entrenamiento.



(a) Convergencia en la época 1 **(b)** Pérdida global (30 épocas)

Figura 4.3: Evolución de la pérdida (*loss*) del modelo MFT-FiLM. Izquierda: rápida convergencia durante las iteraciones iniciales. Derecha: pérdida de entrenamiento promedio frente a la pérdida global de validación (calculada como la suma de los errores de las cuatro subtareas).

Como se aprecia en la Figura 4.3a, el modelo exhibe una convergencia inicial excepcionalmente rápida. Durante las primeras iteraciones de la primera época, la función de pérdida experimenta un descenso abrupto, alcanzando una estabilización prematura. Esta elevada eficiencia en la optimización inicial sugiere que el mecanismo de enrutamiento basado en FiLM (*Feature-wise Linear Modulation*) facilita una adaptación veloz a las representaciones espectro-temporales de la señal de audio. Al analizar la tendencia macroscópica (Figura 4.3b), se observa que la pérdida de entrenamiento continúa decreciendo de forma asintótica, reduciéndose de 0,277 en la época 1 a 0,063 en la época 30. La métrica de validación acompaña esta tendencia de descenso progresivo sin mostrar divergencias, lo cual permite descartar la presencia de sobreajuste (*overfitting*) en esta etapa del entrenamiento.

Continuamos analizando métricas en validación. La Figura 4.4 desglosa la evolución de la métrica F_1 sobre el conjunto de validación para las cuatro subtareas evaluadas. El aprendizaje presenta una estructura jerárquica bien definida: la detección de *Note Onset* alcanza valores de convergencia superiores al 97 % de forma muy temprana (época 5). En contraste, la estimación de la velocidad (*Velocity*), al constituir un problema de regresión dependiente de la intensidad dinámica de la pulsación, requiere un proceso de optimización más prolongado, evolucionando desde un 72,22 % inicial hasta consolidarse por encima del 85 % hacia la culminación del entrenamiento.

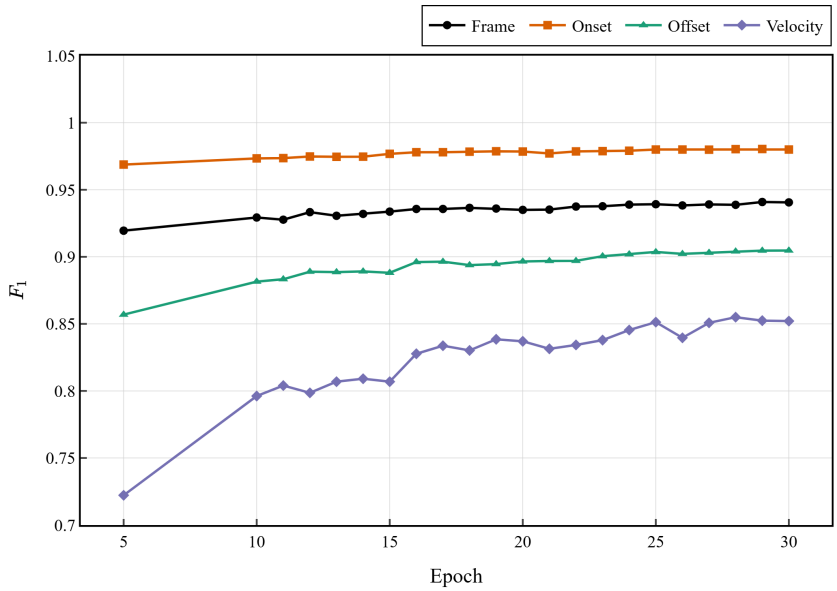
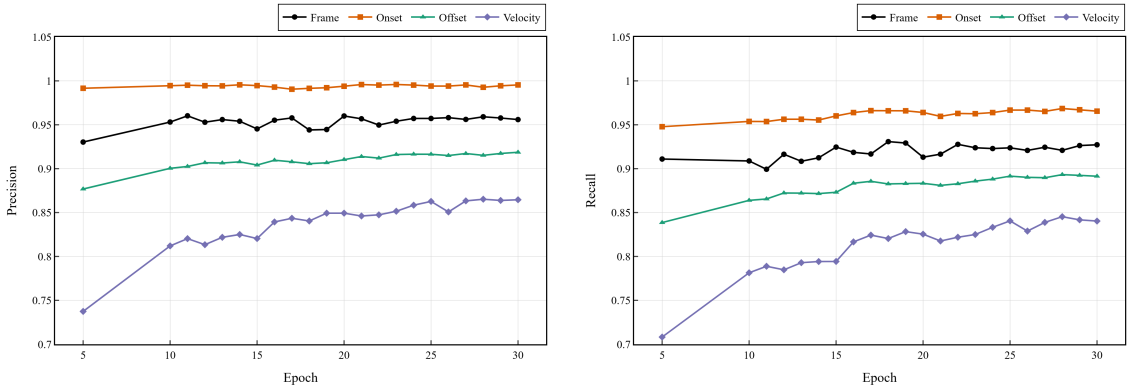


Figura 4.4: Evolución de la métrica F_1 en el conjunto de validación para la estimación de *Frame*, *Onset*, *Offset* y *Velocity* durante las 30 épocas de entrenamiento.

Para analizar rigurosamente la naturaleza de las predicciones del modelo, resulta necesario descomponer esta métrica armónica en sus constituyentes básicos. La Figura 4.5 presenta en paralelo los niveles de precisión y *recall* obtenidos durante la fase de validación.



(a) Precisión en validación

(b) *Recall* en validación

Figura 4.5: Comparativa entre los niveles de precisión y *recall* en el conjunto de validación. Destaca la sistemática superioridad de la precisión frente al *recall* .

El contraste analítico entre ambas gráficas evidencia que el modelo MFT-FiLM presenta un comportamiento marcadamente **conservador**. A lo largo de todas las épocas y

tareas evaluadas, los niveles de precisión (Figura 4.5a) se mantienen de forma sistemática por encima de los niveles de *recall* (Figura 4.5b). A modo de ejemplo, en la época 30, la precisión para la detección de *Note Onset* alcanza valores cuasi perfectos ($> 99{,}5\,\%$), mientras que la exhaustividad (*recall*) se estabiliza en torno al 96,5 %.

Estadísticamente, esta divergencia indica que el modelo penaliza con mayor severidad los falsos positivos frente a los falsos negativos. En el contexto específico de la Transcripción Automática de Música (AMT), esto se traduce en que, ante la predicción de activación de una nota, existe una elevada probabilidad de que dicha nota esté efectivamente presente en la interpretación original. En consecuencia, los errores residuales del sistema provienen mayoritariamente de la omisión de notas (*missed notes*), fenómeno particularmente predecible en segmentos de alta densidad polifónica o ante pulsaciones de baja amplitud acústica enmascaradas por el uso del pedal de *sustain*.

Desde una perspectiva de percepción musical, este sesgo algorítmico resulta altamente favorable. En la reconstrucción auditiva de una partitura transcrita (por ejemplo, mediante la síntesis de los archivos MIDI resultantes), la inserción de notas incorrectas o inexistentes (falsos positivos) genera un impacto acústico muy disruptivo y disonante para el oyente. Por el contrario, la omisión puntual de una nota interna dentro de un clúster armónico complejo (falso negativo) tiende a resultar perceptualmente imperceptible, garantizando así la preservación de la coherencia tonal y rítmica de la obra.

CAPÍTULO 5
Transkun

5.1 Introducción

La transcripción automática de música de piano puede abordarse desde múltiples perspectivas metodológicas. El enfoque más extendido en la literatura consiste en trabajar a nivel de *frame*: el sistema produce, para cada instante de la señal de audio, una estimación de la activación de cada tecla del piano, del inicio y del final de cada nota (*onset* y *offset*), así como de la velocidad de pulsación (*velocity*) [5, 8]. Estas estimaciones se obtienen de forma independiente para cada subtarea y, a continuación, se ensamblan mediante algoritmos heurísticos de postprocesado (umbralización, búsqueda de picos, modelos de Markov de bajo orden) para recuperar los eventos de nota como intervalos discretos [18]. Esta estrategia de dos etapas presenta limitaciones estructurales de fondo: la calidad del resultado a nivel de nota queda fundamentalmente limitada por la calidad de las activaciones *frame* a *frame*, que pueden ser discontinuas, irregulares en magnitud o contradictorias entre subtareas; y el postprocesado manual no forma parte del proceso de optimización, por lo que el sistema no puede aprender a corregir sus propios errores de ensamblado.

El modelo **MFT-FiLM**, estudiado en el capítulo anterior, responde a estas limitaciones desde la perspectiva del aprendizaje multitarea: mediante módulos de modulación lineal entre ramas (*Inter-Task FiLM*), el sistema aprende a intercambiar información entre las subtareas de *onset*, *frame* y *velocity* durante el propio proceso de estimación *frame* a *frame* [11]. Sin embargo, la representación final del resultado sigue siendo un conjunto de predicciones independientes por trama, que requiere un paso de decodificación posterior para construir los eventos de nota.

El segundo modelo implementado en este trabajo, **Transkun** (*Transcription Using Neural semi-CRFs*), propone una solución radicalmente diferente: eliminar por completo el nivel de *frame* como objetivo de optimización, y sustituirlo por la predicción directa de intervalos de tiempo. La idea central consiste en tratar cada nota como un intervalo cerrado $[i, j]$ sobre el eje temporal, y asignar a cada candidato de intervalo una puntuación escalar que refleje su probabilidad de corresponder a un evento real del tipo de evento dado [16]. Esta reformulación del problema se apoya en el **campo aleatorio condicional semi-Markoviano** (*neural semi-Markov Conditional Random Field*, semi-CRF), un marco de predicción estructurada que permite modelar directamente conjuntos de intervalos no solapados y decodificarlos de forma óptima mediante programación dinámica [15].

La familia de modelos basada en semi-CRF para la transcripción de piano fue introducida por Yan, Cwitkowitz y Duan [15], quienes demostraron que este enfoque podía superar los métodos *frame-level* en métricas de nota holística (*Note w/ Offset*, *Note w/ Offset & Velocity*), al tiempo que presentaba una complejidad de inferencia tratable. La arqui-

tectura original empleaba un modelo contextual basado en bloques convolucionales y redes recurrentes GRU bidireccionales para extraer representaciones por trama, y un módulo MLP para puntuar cada candidato de intervalo a partir de los embeddings de sus extremos y de sus momentos estadísticos internos [15].

Transkun [16], propuesto por Yan y Duan en ISMIR 2024, construye sobre esta base e introduce dos contribuciones fundamentales que simplifican y mejoran el marco original. En primer lugar, reemplaza el módulo de *scoring* MLP por un **producto interior escalado** entre vectores de representación de inicio y fin del intervalo (una operación análoga al mecanismo de atención de los transformers), y demuestra teóricamente que, bajo una condición de rango moderada, esta operación es suficientemente expresiva para representar cualquier matriz de *scoring* ideal [16]. En segundo lugar, aprovecha la analogía formal entre el producto interior propuesto y la atención *transformer* para adoptar como *backbone* un **transformer encoder no jerárquico** que opera sobre un mapa de características de baja resolución temporal, sin preservar la resolución de la entrada en todos los niveles como hacen los modelos SOTA anteriores. Este diseño, inspirado en la literatura de Vision Transformers [17], reduce el coste computacional y mejora la escalabilidad del modelo sin sacrificar precisión en la localización de eventos [16].

El resultado es un sistema de un solo stage que produce, directamente a partir del espectrograma log-mel de entrada, un conjunto óptimo de intervalos no solapados para cada uno de los 88 tipos de nota y los pedales del piano, sin necesidad de postprocesado heurístico. Con 12,9 millones de parámetros, Transkun establece el nuevo estado del arte en todas las subtareas de transcripción sobre el conjunto de test de MAESTRO v3 en términos de puntuación F_1 *offset* y *velocity* [16].

En este capítulo, la Sección 5.2 describe la arquitectura completa del sistema, desde el espectrograma de entrada hasta el transformer encoder y los *event tracks*. La Sección 5.3 presenta el marco semi-CRF y la formulación del scoring por producto interior escalado. La Sección 5.4 detalla la predicción de atributos y el procesado por segmentos. A continuación, la Sección 5.5 expone el entorno de cómputo y la configuración experimental del entrenamiento. En la Sección 5.6 se describe el preprocesamiento del *dataset* y el desarrollo práctico del proceso de entrenamiento en el clúster. Finalmente, en la Sección 5.7 se presentan y analizan los resultados obtenidos, comparando el rendimiento con el estado del arte y con el modelo MFT-FiLM estudiado en el capítulo anterior.

5.2 Arquitectura del modelo

La arquitectura de Transkun está diseñada para producir, a partir del espectrograma de audio de entrada, un conjunto de representaciones vectoriales que permitan puntuar cada posible intervalo temporal mediante un simple producto interior. El sistema puede dividirse en cuatro etapas funcionales: la representación tiempo-frecuencia de la señal, el submuestreo y proyección a un mapa de características de baja resolución, el procesado contextual mediante un transformer encoder no jerárquico, y el remuestreo final para recuperar la resolución temporal original. En la Figura 5.1 se presenta el diagrama general del sistema.

Representación de entrada: espectrograma log-mel sobruestreado

La señal de audio de entrada se muestrea a 44 100 Hz y se transforma en un espectrograma logarítmico mediante la Transformada de Fourier de Tiempo Corto (al igual que MFT-FiLM, Sección 4.2), con un tamaño de ventana de 4096 muestras, un salto de trama

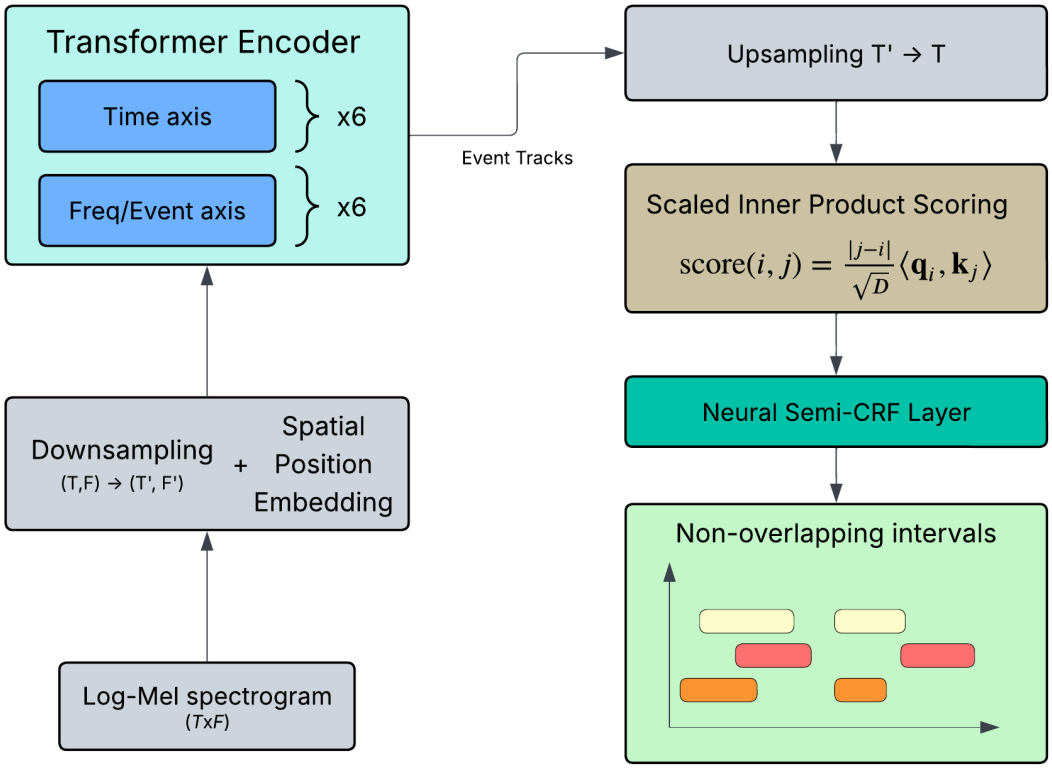


Figura 5.1: Esquema general del sistema TranskUN. El espectrograma log-mel es submuestreado mediante convoluciones *strided* y enriquecido con embeddings de posición espacial. Los *event tracks* se concatenan a las representaciones del espectrograma y se procesan conjuntamente por el transformer encoder. Tras el remuestreo temporal, los *event tracks* resultantes se utilizan para el scoring por producto interior, que alimenta la capa semi-CRF. Ilustración adaptada de [16].

(*hop size*) de 1024 muestras y la ventana de Hanning. Se utilizan 229 bandas en el rango de 30 a 8000 Hz [16].

Para mejorar la resolución temporal de la representación espectral, se aplica una estrategia de **sobruestreo de ventanas** (*multi-window oversampling*), heredada del modelo semi-CRF original [15]: en lugar de utilizar una única ventana de Hanning por trama, se aplican cinco subventanas de Hanning ajustables (inicializadas con centros equidistantes y varianza constante) sobre cada trama de audio. Los espectros log-mel resultantes se concatenan a lo largo del eje de canales, produciendo una entrada de cinco canales espectrales por *bin* tiempo-frecuencia. De este modo, la resolución efectiva del espectrograma en el eje temporal aumenta sin necesidad de reducir el salto de trama, lo que mantiene la eficiencia computacional del preprocesado. Cada segmento de audio procesado tiene una duración de 16 segundos [16].

Submuestreo y embeddings de posición espacial

A diferencia de los sistemas SOTA previos (incluyendo el propio semi-CRF original [15], hFT-Transformer [13] y Onsets & Frames [5]), que preservan la resolución temporal completa de la entrada a lo largo de toda la red, Transkun adopta un diseño de **mapa de características de baja resolución**, análogo al empleado en los Vision Transformers (ViT) [17]. Este enfoque, que en visión artificial ha demostrado ser eficaz incluso para tareas de predicción densa [22], permite reducir drásticamente el coste computacional del

transformer sin sacrificar la capacidad expresiva del modelo, siempre que la dimensionalidad de los embeddings sea suficientemente alta.

El espectrograma de entrada, de dimensiones $(T \times F)$, se submuestrea mediante **cuatro capas convolucionales *strided*** con kernel de tamaño 3 y canales de salida $[128, 256, 256, 256]$ respectivamente. Los *strides* aplicados son $[(2,1), (2,2), (2,2), (1,1)]$ sobre los ejes (tiempo y frecuencia), lo que produce un factor de submuestreo de $c_T = 8$ en tiempo y de $c_F = 4$ en frecuencia. El resultado es un mapa de características de baja resolución de dimensiones $(T', F') = (T/8,\ F/4)$, equivalente a parches de tamaño 8×4 sobre el espectrograma original [16]. Cada capa convolucional va seguida de una normalización **GroupNorm** con 4 grupos y una activación GELU, a excepción de la última capa.

Tras el submuestreo, se añaden **embeddings de posición espacial aprendibles** basados en características de Fourier (*learnable Fourier features*) [21], tanto a las representaciones tiempo-frecuencia del espectrograma (con coordenadas $(\mathit{frameIdx}, \mathit{freqIdx})$) como a los *event tracks* (con coordenadas $(\mathit{frameIdx}, \mathit{eventTypeIdx})$) [16]. Este embedding se computa como:

$$\boldsymbol{y} = g\left(\sqrt{\tfrac{2}{B}}\,\cos(\boldsymbol{W}_r\,\boldsymbol{x} + \boldsymbol{b})\right) \tag{5.1}$$

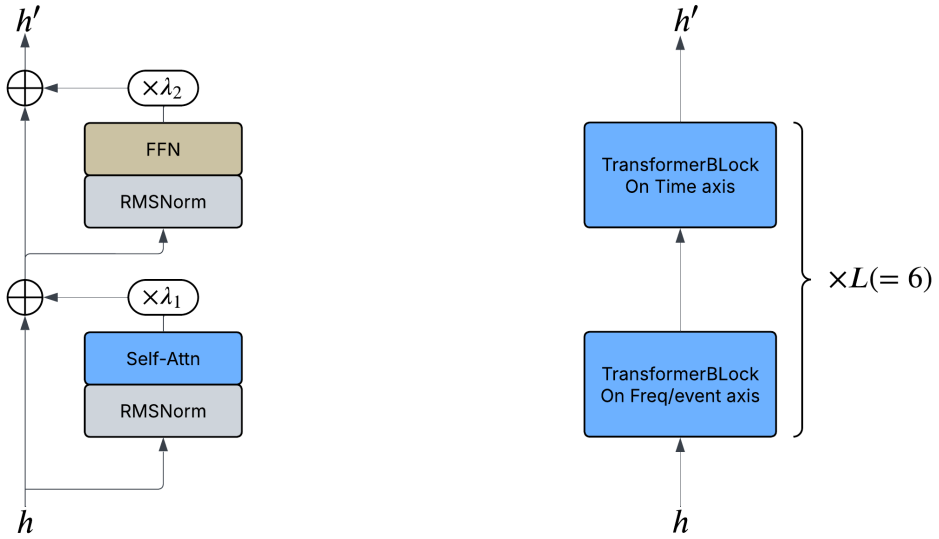
donde $\boldsymbol{W}_r \in \mathbb{R}^{B\times C}$ es una matriz aprendible inicializada desde $\mathcal{N}(0,\ \gamma^{-2})$; $\boldsymbol{b} \in \mathbb{R}^B$ es un sesgo aprendible inicializado desde $\mathcal{U}(-\pi,\ +\pi)$; B es la dimensión de las características de Fourier; γ es un hiperparámetro de escala; y $g : \mathbb{R}^B \to \mathbb{R}^E$ es un perceptrón de dos capas [16]. Este embedding funciona como una red MLP que toma coordenadas espaciales como entrada, con la primera no linealidad siendo una función coseno escalada. Se elige por su sencillez y amplia compatibilidad con arquitecturas transformer [16].

Event tracks y concatenación

Para obtener las representaciones necesarias para el scoring de intervalos, el sistema introduce el concepto de ***event track***: una secuencia de vectores de baja resolución temporal, uno por tipo de evento (88 notas + pedales), que actuará como representación aprendible de cada tipo de evento a lo largo del tiempo. Estos *event tracks* se inicializan con sus propios embeddings de posición espacial y se **concatenan** a las representaciones del espectrograma submuestreado a lo largo de la dimensión de secuencia. De este modo, el transformer encoder recibe como entrada una secuencia mixta que combina las representaciones tiempo-frecuencia del audio y los *event tracks* de todos los tipos de evento, y puede intercambiar información entre ambas a través del mecanismo de autoatención [16].

Transformer encoder no jerárquico

El transformer encoder es el componente central del sistema. Se trata de un **encoder no jerárquico** (sin submuestreo adicional ni estructura piramidal) compuesto por $L = 6$ capas, cada una de las cuales contiene dos bloques transformer en cascada: uno que atiende a lo largo del **eje temporal** y otro a lo largo del **eje de frecuencia o tipo de evento**, alternando en cada capa para reducir el coste computacional [16]. Esta estrategia de atención axial separable es análoga a la empleada en otras arquitecturas transformer eficientes para datos multidimensionales [20, 23].



(a) TransformerBlock **(b)** Encoder

Figura 5.2: Bloques constitutivos del transformer encoder. (a) Bloque transformer básico con pre-normalización RMSNorm y conexión residual escalada mediante ReZero. (b) Organización de los bloques en cada capa del encoder, alternando la atención sobre el eje temporal y el eje de frecuencia o tipo de evento. Ilustración adaptada de [16].

Cada bloque transformer sigue el diseño estándar de *pre-normalización*: primero se aplica **RMSNorm** [24] sobre la entrada, después la capa de auto-atención o de red neuronal de avance (*feed-forward network*, FFN con tamaño 1024), y finalmente la salida se añade a la conexión residual. Para mejorar la estabilidad durante el entrenamiento de modelos profundos, se emplea **ReZero** [19]: un factor de escala aprendible λ, inicializado a 0,01, multiplica la salida de cada sublcapa antes de sumarse al residual. Este mecanismo permite que, al inicio del entrenamiento, la salida de cada bloque sea casi idéntica a su entrada, facilitando el flujo del gradiente y acelerando la convergencia. El encoder utiliza 8 cabezas de atención y una dimensión de embedding de 256 [16].

Remuestreo temporal

Tras el procesado por el transformer encoder, solo se retienen los *event tracks* (descartando las representaciones del espectrograma), ya que son estos vectores los que se utilizarán para el scoring de intervalos. Dado que los *event tracks* operan en la resolución temporal reducida ($T' = T/8$), es necesario recuperar la resolución original para asignar puntuaciones a intervalos con precisión de trama.

Para ello se aplica una única **capa convolucional transpuesta 1D** (`ConvTranspose1d`) con tamaño de kernel y *stride* igual a 8, 128 canales de salida y sin activación no lineal. Esta capa realiza un remuestreo **lineal** a lo largo del eje temporal que, como se demuestra en el Teorema 3.2 del artículo [16] (discutido en detalle en la Sección 5.3), es suficiente para preservar la expresividad del operador de scoring siempre que la dimensionalidad D' del espacio de baja resolución supere el producto $c \cdot |\mathcal{Y}|$, donde $c = 8$ es el factor de remuestreo y $|\mathcal{Y}|$ es el número de intervalos activos. Los *event tracks* resultantes tienen dimensiones $(T \times D)$ con $D = 128$, y constituyen la entrada al módulo de scoring de intervalos descrito en la sección siguiente.

5.3 *Scoring* por producto interior y semi-CRF

El núcleo matemático de Transkun reside en dos elementos: el **campo aleatorio condicional semi-Markoviano** (*neural semi-Markov Conditional Random Field*, semi-CRF), que proporciona el marco de predicción estructurada para modelar conjuntos de intervalos no solapados; y el ***scoring* por producto interior escalado**, que constituye el método para asignar una puntuación a cada intervalo candidato de forma eficiente y teóricamente fundamentada [16]. En esta sección se describe primero el *framework* semi-CRF y su formulación probabilística, y a continuación se presenta el método de *scoring* propuesto, su justificación teórica y su relación con el mecanismo de atención de los transformers.

El *framework* neural semi-CRF

Para una secuencia de T tramas de audio, el objetivo de la transcripción puede formularse como la inferencia de un conjunto $\mathcal{Y}$ de intervalos cerrados no solapados de la forma $[i, j]$, donde i y j son índices de trama de inicio y fin respectivamente, y cada intervalo se asocia a un tipo de evento específico (una nota concreta o un pedal). El semi-CRF modela directamente esta estructura de salida definiendo, para cada tipo de evento, dos tipos de puntuaciones a partir de la señal de audio $\mathcal{X}$ [15, 16]:

1. $\text{score}(i, j)$: una matriz triangular $T \times T$ que asigna una puntuación a cada intervalo candidato $[i, j]$, indicando en qué medida dicho intervalo corresponde a un evento activo del tipo de evento dado. Los valores diagonales $\text{score}(i, i)$ representan eventos de trama única.
2. $\text{score}_\epsilon(i-1, i)$: un vector de dimensión $T-1$ que asigna una puntuación a cada par de tramas consecutivas $[i-1, i]$ no cubiertas por ningún intervalo en $\mathcal{Y}$, representando la inactividad del tipo de evento en ese instante.

La puntuación total de un conjunto de intervalos $\mathcal{Y}$ dada la entrada $\mathcal{X}$ se define como [16]:

$$\Phi(\mathcal{Y} \mid \mathcal{X}) = \sum_{[i,j] \in \mathcal{Y}} \text{score}(i, j) \; + \sum_{\substack{[i-1,i] \\ \text{no cubierto en } \mathcal{Y}}} \text{score}_\epsilon(i-1, i) \tag{5.2}$$

Para la **inferencia**, se utiliza la estimación del máximo a posteriori (MAP), que recupera el conjunto óptimo de intervalos no solapados mediante programación dinámica (algoritmo de Viterbi):

$$\mathcal{Y}^* = \arg\max_{\mathcal{Y}} \; \Phi(\mathcal{Y} \mid \mathcal{X}) \tag{5.3}$$

Para el **entrenamiento**, se emplea estimación de máxima verosimilitud (MLE), con la log-verosimilitud condicional definida como:

$$\log p(\mathcal{Y} \mid \mathcal{X}) = \Phi(\mathcal{Y} \mid \mathcal{X}) - \log \sum_{\mathcal{Y}'} \exp \; \Phi(\mathcal{Y}' \mid \mathcal{X}) \tag{5.4}$$

donde la suma en el segundo término recorre todos los posibles conjuntos de intervalos no solapados. Este término de normalización, equivalente a la función de partición del modelo, se computa eficientemente mediante un algoritmo *forward-backward* [15]. Se instancia un semi-CRF independiente para cada tipo de evento, procesando en paralelo los 88 tipos de nota y los pedales.

Scoring por producto interior escalado

El módulo de *scoring* es el componente responsable de calcular la matriz $\text{score}(i, j)$ a partir de las representaciones del modelo. En el semi-CRF original [15], este módulo empleaba un MLP que tomaba como entrada los *embeddings* de los extremos del intervalo $\boldsymbol{h}_i$ y $\boldsymbol{h}_j$, su producto de Hadamard $\boldsymbol{h}_i \odot \boldsymbol{h}_j$ y los tres primeros momentos estadísticos sobre el intervalo:

$$\text{score}(i, j) = \text{MLP}([\boldsymbol{h}_i,\, \boldsymbol{h}_j,\, \boldsymbol{h}_i \odot \boldsymbol{h}_j,\, \boldsymbol{m}_1,\, \boldsymbol{m}_2,\, \boldsymbol{m}_3]) \tag{5.5}$$

Este enfoque resulta costoso en memoria, ya que la matriz de *scoring* crece cuadráticamente con la longitud de la secuencia, y la evaluación del MLP para cada par (i, j) requiere procesamiento por segmentos y *gradient checkpointing* durante el entrenamiento, lo que limita la capacidad y profundidad del módulo [16].

Transkun propone sustituir este módulo por una operación de **producto interior escalado** entre vectores de baja dimensión:

$$\text{score}(i, j) = \frac{|j - i|}{\sqrt{D}} \langle \boldsymbol{q}_i,\, \boldsymbol{k}_j \rangle + b_i\, \delta(i, j) \tag{5.6}$$

donde $\delta(i, j)$ es la delta de Kronecker, que vale 1 si $i = j$ y 0 en caso contrario. Los vectores $\boldsymbol{q}_i \in \mathbb{R}^D$, $\boldsymbol{k}_i \in \mathbb{R}^D$ y el escalar $b_i \in \mathbb{R}$ se obtienen directamente del *event track* $\boldsymbol{h}_i$ mediante una única capa lineal f:

$$[\boldsymbol{q}_i,\, \boldsymbol{k}_i,\, b_i] = f(\boldsymbol{h}_i) \tag{5.7}$$

La matriz de *scoring* resultante tiene una estructura de **rango bajo más diagonal**: el término del producto interior produce una matriz de rango D, y el término diagonal $b_i\, \delta(i, j)$ añade los valores de la diagonal principal de forma independiente. Esta formulación, denominada *Scaled Inner Product Interval Scoring*, es notablemente más simple que el MLP original y no requiere calcular momentos estadísticos sobre el intervalo [16].

Una pregunta natural es si la simplicidad del producto interior implica una pérdida de capacidad expresiva respecto al MLP original. Yan y Duan demuestran formalmente que, bajo una condición moderada sobre el rango de las representaciones, el producto interior es suficientemente expresivo para representar cualquier matriz de *scoring* ideal [16].

Para formalizar esto, se define una **matriz de *scoring* ideal** $\boldsymbol{S}_{\mathcal{Y}} \in \mathbb{R}^{T \times T}$ para un conjunto de intervalos $\mathcal{Y}$ como aquella tal que:

$$\boldsymbol{S}_{\mathcal{Y}}(i, j) > 0 \quad \forall\, [i, j] \in \mathcal{Y}, \qquad \boldsymbol{S}_{\mathcal{Y}}(i, j) = -\epsilon \quad \text{en caso contrario}$$

con $\epsilon > 0$. Con esta matriz, la decodificación MAP produce necesariamente $\mathcal{Y}$, ya que excluir cualquier intervalo activo o incluir cualquier intervalo inactivo reduciría la puntuación total.

Lema (Yan y Duan, 2024)**.** *El rango de una matriz de scoring ideal $\boldsymbol{S}_{\mathcal{Y}}$ para un conjunto de intervalos no solapados $\mathcal{Y}$ es $M + 1$, donde $M = |\mathcal{Y}|$ es el número de intervalos activos.*

Teorema (Yan y Duan, 2024)**.** *Sea $\mathcal{Y}$ un conjunto de M intervalos no solapados sobre T pasos. Una matriz de scoring ideal $\boldsymbol{S}_{\mathcal{Y}}$ puede representarse como el producto interior por pares entre dos secuencias 1D $(\boldsymbol{k}_i)_i$ y $(\boldsymbol{q}_i)_i$ de vectores:*

$$\boldsymbol{S}_{\mathcal{Y}}(i, j) = \langle \boldsymbol{q}_i, \boldsymbol{k}_j \rangle$$

con $\text{rank}(\boldsymbol{Q}_{\mathcal{Y}}) > M$ y $\text{rank}(\boldsymbol{K}_{\mathcal{Y}}) > M$, donde $\boldsymbol{Q}_{\mathcal{Y}} = [\boldsymbol{q}_0, \ldots, \boldsymbol{q}_{T-1}]$ y $\boldsymbol{K}_{\mathcal{Y}} = [\boldsymbol{k}_0, \ldots, \boldsymbol{k}_{T-1}]$.

Este resultado tiene dos implicaciones prácticas directas [16]. En primer lugar, la dimensión D de los vectores $\boldsymbol{q}_i$ y $\boldsymbol{k}_i$ debe superar el número total de intervalos activos $|\mathcal{Y}|$. En segundo lugar, si los vectores se obtienen aplicando un operador de remuestreo lineal u_c sobre una secuencia de baja resolución temporal (como hace la capa `ConvTranspose1d` del sistema), la dimensión requerida en el espacio de baja resolución es $D' > c \cdot |\mathcal{Y}|$, donde $c = T/T'$ es el factor de remuestreo. Esto garantiza que operar sobre el mapa de características submuestreado no implica ninguna pérdida de expresividad, siempre que la dimensión de los embeddings sea suficientemente alta.

Relación con el mecanismo de atención

La analogía entre el *scoring* propuesto en la Ecuación 5.6 y el mecanismo de atención por producto interior escalado de los transformers [25] es inmediata: ambos computan una puntuación escalar para cada par de posiciones (i, j) de una secuencia mediante el producto interior entre un vector de consulta $\boldsymbol{q}_i$ y un vector de clave $\boldsymbol{k}_j$, escalado por $1/\sqrt{D}$. Sin embargo, existen diferencias estructurales relevantes. La atención normaliza las puntuaciones por posición mediante una operación *softmax*, lo que convierte cada fila de la matriz en una distribución de probabilidad local. El semi-CRF, en cambio, normaliza de forma **global** sobre todos los posibles conjuntos de intervalos no solapados $\mathcal{Y}'$, lo que garantiza que la decodificación sea globalmente óptima en lugar de localmente *greedy* [16].

Esta analogía no es meramente formal: motiva directamente la elección del transformer como *backbone* para producir las representaciones $(\boldsymbol{h}_i)$ que alimentan el módulo de *scoring*. El transformer puede interpretarse como una arquitectura que refina iterativamente las representaciones de una secuencia precisamente para que sean adecuadas para el *scoring* por producto interior, lo que establece una coherencia entre la arquitectura del modelo y el mecanismo de salida [16].

Por último, cabe señalar que el estudio de ablación presentado en el artículo original demuestra que una capa lineal para el cálculo de los vectores $\boldsymbol{q}$, $\boldsymbol{k}$ y b (Ecuación 5.7) obtiene mejores resultados que un MLP, lo que refuerza la analogía con los transformers, donde $\boldsymbol{q}$ y $\boldsymbol{k}$ se calculan también mediante proyecciones lineales [16].

5.4 Predicción de atributos y procesado por segmentos

La capa semi-CRF produce, tras la decodificación MAP, un conjunto de intervalos $[i, j]$ para cada tipo de evento, que especifican el inicio y el fin de cada nota o pedal con precisión de trama. Sin embargo, la representación MIDI final requiere información adicional que no está codificada en los índices de trama: la **velocidad** de cada nota, las **posiciones subframe** de inicio y fin para dequantizar los instantes exactos de onset y offset, y un par de indicadores binarios para gestionar correctamente los eventos que cruzan los límites entre segmentos de audio consecutivos. En esta sección se describen el módulo de predicción de estos atributos y la estrategia de procesado por segmentos que permite transcribir grabaciones de duración arbitraria.

Predicción de atributos por evento

Una vez decodificados los intervalos, cada evento $[a, b]$ de un tipo de evento concreto se caracteriza mediante cuatro atributos, predichos por un **MLP de dos capas** con tamaño

oculto de 512 unidades y *dropout* de tasa 0,1 [16]. La entrada al MLP son los embeddings del *event track* en los extremos del intervalo, $\boldsymbol{h}_a^{\text{eventType}}$ y $\boldsymbol{h}_b^{\text{eventType}}$, concatenados:

- **Velocidad** (*velocity*) $\in \{0, \ldots, 127\}$: modelada como una distribución categórica sobre 128 clases mediante una capa *softmax*. La velocidad MIDI representa la intensidad de pulsación de cada tecla y es un valor entero en el rango estándar MIDI [16].
- **Posiciones refinadas de onset y offset**: dado que los índices de trama a y b son posiciones cuantizadas (cada trama tiene una duración de aproximadamente 23,2 ms con un *hop size* de 1024 muestras a 44 100 Hz), los instantes exactos de inicio y fin de cada nota pueden caer en cualquier punto dentro de esa trama. Para recuperar esta información subframe, el modelo predice un valor $\in (-0{,}5,\ 0{,}5)$ para cada extremo, que indica el desplazamiento relativo respecto al índice cuantizado. Estos valores se modelan mediante una **distribución de Bernoulli continua** desplazada en $-0{,}5$ [26], que proporciona una distribución de probabilidad bien definida sobre el intervalo abierto $(-0{,}5,\ 0{,}5)$ con soporte continuo y gradientes no nulos en todo el dominio [16].
- **Indicadores** `hasOnset` **y** `hasOffset` $\in \{0, 1\}$: dos variables binarias modeladas mediante distribuciones de Bernoulli, cuyo significado se detarán a continuación.

La pérdida de entrenamiento del módulo de predicción de atributos se suma a la log-verosimilitud del semi-CRF (Ecuación 5.4), de forma que el objetivo total es [16]:

$$\mathcal{L} = -\left[\log p(\mathcal{Y} \mid \mathcal{X}) + \sum_{e \in \mathcal{Y}} \log p(\text{atributos} \mid e)\right] \tag{5.8}$$

donde el segundo término se descompone en:

$$\begin{aligned} \log p(\text{atributos} \mid e) = \ & \log p(\text{velocity} \mid e) \\ & + \log p(\text{onset refinado} \mid e) \\ & + \log p(\text{offset refinado} \mid e) \end{aligned} \tag{5.9}$$

Procesado por segmentos y gestión de eventos incompletos

El transformer encoder opera sobre segmentos de audio de duración fija de **16 segundos**. Para transcribir grabaciones de mayor duración, el audio se divide en segmentos consecutivos con un **solapamiento del 50 %** (8 segundos de solape entre segmentos adyacentes), y los resultados de cada segmento se ensamblan en una secuencia global de eventos [16].

El principal desafío de esta estrategia es la gestión de los eventos que **cruzan el límite** de un segmento: una nota larga que comienza en el segmento k y termina en el segmento $k+1$ no puede representarse como un único intervalo $[i, j]$ dentro de ninguno de los dos segmentos por separado. El semi-CRF original de Yan *et al.* [15] resolvía este problema descartando directamente todos los eventos que excedían el límite del segmento durante el entrenamiento. TranskUN adopta una solución más precisa: en lugar de descartar, **trunca** el evento en el límite del segmento e introduce dos indicadores binarios por evento para registrar si el truncamiento ha tenido lugar [16]:

- `hasOnset` $= 1$: el onset del evento pertenece al segmento actual (no ha sido truncado por el límite izquierdo).

- `hasOffset` $= 1$: el offset del evento pertenece al segmento actual (no ha sido truncado por el límite derecho).

Con estos indicadores, el algoritmo de decodificación por segmentos sigue la siguiente lógica para cada tipo de evento [16]:

1. El proceso de decodificación dentro de cada segmento comienza desde el máximo entre: (a) el límite izquierdo del segmento actual, y (b) el offset del último evento en el conjunto resultado con `hasOffset` $= 1$.
2. Los eventos decodificados con `hasOnset` $= 1$ que no se solapan con el último evento del conjunto resultado se añaden directamente.
3. Para los eventos que se solapan con el último evento del conjunto resultado:
   - Si el nuevo evento tiene `hasOnset` $= 1$, reemplaza al anterior (el anterior era un fragmento truncado que ya no tiene soporte en el nuevo segmento).
   - Si el nuevo evento tiene `hasOnset` $= 0$, se trata de la continuación del evento anterior, y ambos fragmentos se **fusionan** en un único evento.

Este mecanismo es especialmente relevante para los **pedales**, que con frecuencia superan los 16 segundos de duración de un segmento (en particular el pedal *sustain*), mientras que las notas individuales raramente exceden este límite. El estudio de ablación presentado en el artículo original confirma que omitir el tratamiento de eventos incompletos produce una degradación notable en las métricas de pedal pero tiene un impacto prácticamente nulo en las métricas de notas [16].

5.5 Configuración del experimento

5.6 Preproceso y entrenamiento

5.7 Resultados

CAPÍTULO 6
Conclusiones

????? ????????????? ????????????? ????????????? ????????????? ?????????????

Bibliografía

[1] Valentin Emiya, Nancy Bertin, Bertrand David y Roland Badeau. MAPS - A piano database for multipitch estimation and automatic transcription of music. *INRIA Technical Report*, 2010.

[2] Francesco Foscarin, Katharina Hoedt, Gerhard Widmer y Maarten Grachten. Towards musically informed evaluation of piano transcription models. *arXiv preprint arXiv:2406.08454*, 2024.

[3] Google Magenta. The MAESTRO *dataset* v3.0.0. 2020.

[4] Yamaha Corporation. *Disklavier pianos.* Disponible en: https://usa.yamaha.com/products/musical_instruments/pianos/disklavier/index.html Consultado en: mayo de 2026.

[5] Curtis Hawthorne, Erich Elsen, Jialin Song, Adam Roberts, Ian Simon, Colin Raffel, Jesse Engel, Sageev Oore y Douglas Eck. Onsets and frames: Dual-objective piano transcription. *Proceedings of the International Society for Music Information Retrieval (ISMIR)*, pp. 50–57, 2018.

[6] Curtis Hawthorne, Andriy Stasyuk, Adam Roberts, Ian Simon, Cheng-Zhi Anna Huang, Sander Dieleman y Douglas Eck. Enabling factorized piano music modeling and generation with the MAESTRO *dataset*. *International Conference on Learning Representations (ICLR)*, 2019.

[7] Colin Raffel, Brian McFee, Eric J. Humphrey, Justin Salamon, Oriol Nieto, Dawen Liang y Daniel P. W. Ellis. mir_eval: A transparent implementation of common MIR metrics. *Proceedings of the 15th International Conference on Music Information Retrieval*, 2014.

[8] Qiuqiang Kong, Bochen Li, Xuchen Song, Yuan Wan y Yuxuan Wang. High-resolution piano transcription with pedals by regressing onset and offset times. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3707–3717, 2021.

[9] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin y Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

[10] Qi Wang, Mingkuan Liu, Changchun Bao y Maoshen Jia. Harmonic-aware frequency and time attention for automatic piano transcription. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 32, pp. 2140–2152, 2024.

[11] Qi Wang, Mingkuan Liu y Maoshen Jia. Multitask frequency-time feature-wise linear modulation for piano transcription with pedal. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 33, pp. 1049–1062, 2025.

[12] C. Hawthorne, I. Simon, R. Swavely, E. Manilow y J. Engel. Sequence-to-sequence piano transcription with transformers. *Proceedings of the International Society for Music Information Retrieval (ISMIR)*, pp. 246–253, 2021.

[13] K. Toyama, T. Akama, Y. Ikemiya, Y. Takida, W.-H. Liao y Y. Mitsufuji. Automatic piano transcription with hierarchical frequency-time transformer. *Proceedings of the International Society for Music Information Retrieval (ISMIR)*, pp. 1–8, 2023.

[14] W. Wei, P. Li, Y. Yu y W. Li. HPPNet: Modeling the harmonic structure and pitch invariance in piano transcription. *Proceedings of the International Society for Music Information Retrieval (ISMIR)*, pp. 709–716, 2022.

[15] Y. Yan, F. Cwitkowitz y Z. Duan. Skipping the frame-level: Event-based piano transcription with neural semi-CRFs. *Proceedings of the Annual Conference on Neural Information Processing Systems (NeurIPS)*, vol. 25, pp. 20583–20595, 2021.

[16] Y. Yan y Z. Duan. Scoring time intervals using non-hierarchical transformer for automatic piano transcription. *Proceedings of the International Society for Music Information Retrieval (ISMIR)*, pp. 1–8, 2024.

[17] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit y Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *International Conference on Learning Representations (ICLR)*, 2021.

[18] Taegyun Kwon, Dasaem Jeong y Juhan Nam. Polyphonic piano transcription using autoregressive multi-state note model. *Proceedings of the International Society for Music Information Retrieval (ISMIR)*, pp. 454–461, 2020.

[19] T. Bachlechner, B. P. Majumder, H. Mao, G. Cottrell y J. McAuley. ReZero is all you need: Fast convergence at large depth. *Proceedings of the Thirty-Seventh Conference on Uncertainty in Artificial Intelligence (UAI)*, vol. 161, pp. 1352–1361, 2021.

[20] J. Ho, N. Kalchbrenner, D. Weissenborn y T. Salimans. Axial attention in multidimensional transformers. *arXiv preprint arXiv:1912.12180*, 2019.

[21] Y. Li, S. Si, G. Li, C.-J. Hsieh y S. Bengio. Learnable Fourier features for multi-dimensional spatial positional encoding. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[22] Y. Li, H. Mao, R. Girshick y K. He. Exploring plain vision transformer backbones for object detection. *European Conference on Computer Vision (ECCV)*, pp. 280–296, 2022.

[23] N.-C. Ristea, R. T. Ionescu y F. S. Khan. Septr: Separable transformer for audio spectrogram processing. *Proceedings of INTERSPEECH*, pp. 4103–4107, 2022.

[24] B. Zhang y R. Sennrich. Root mean square layer normalization. *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[25] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser e Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[26] G. Loaiza-Ganem y J. P. Cunningham. The continuous Bernoulli: Fixing a pervasive error in variational autoencoders. *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

APÉNDICE A

Configuració del sistema

????? ????????????? ????????????? ????????????? ????????????? ?????????????

A.1 Fase d'inicialització

????? ????????????? ????????????? ????????????? ????????????? ?????????????

A.2 Identificació de dispositius

????? ????????????? ????????????? ????????????? ????????????? ?????????????

APÉNDICE B
??? ????????????? ????

????? ????????????? ????????????? ????????????? ????????????? ?????????????